PromptForge（提示词锻造台）

面向 Coding 场景的提示词编译与多智能体竞演决策 Agent

参赛赛事：首届 openJiuwen 智能体创新大赛（主题「智体共生 · 创新无限」，华为开发者大赛 · 应用创新实践赛）

参赛方向：Coding 赛道

团队：deforoout 队

文档版本：v1.2（创意提交基线：新增机机联动专章）| **日期**：2026-10-30 | **状态**：创意提交 / 开发基线

文档性质：本文件是参赛作品的**设计文档 / 技术报告底稿 / 开发基线**，不是官方创意 PPT；官方创意提交需按模板另行制作 PPT（附录 C 已给出逐页映射），DEMO 阶段以本文档为基础扩展技术报告。配套的《PromptForge 实施工作台 v1.1》用于记录模块接口、开发切片与开放问题。

让 AI 开发从「反复试错」变成「一次选对」

v1.2 创意提交基线 | 2026-10-30

摘要

今天的 AI 开发不缺"能写代码的模型"，缺的是**"一次把方向选对"的方法**：提示词质量方差大、单模型自评有偏差、多方案比较没有证据、来回返工把 token 和时间都烧掉。PromptForge 把"人对着一个大模型反复试错"改造成一条可度量、可复现的流水线：**需求编译（关键词 → 含可执行验收标准的 SRD）→ 多候选提示词（假设不同而非措辞不同）→ 多模型并行执行 → 机器可读证据回读（测试/静态分析/性能/截图/日志）→ 异构多评委盲评 + 可执行验证 → 多准则决策（TOPSIS/Pareto，带置信区间）→ 优点融合与回归 → 预算内收敛交付**。

作品的五个创新点是：**证据驱动的多方案决策、异构评委去偏 + 统计置信、Token 预算感知的自适应编排、Orchestrator + Skill 双形态**（解决"外部 LLM 指挥、执行模型闷头干"的脱节问题）、**机机联动协议 IAOV**（让规划机、执行机、评审机形成可审计的执行闭环，见 3.11）。对 UI、图表、演示视频等非代码产物，系统提供**可选的多模态证据通道**：用关键帧/截图 + OCR/ASR/元数据等工具提取结构化字段，写入证据包后再交给评委（详见 3.3.4）；代码类任务仍以测试、编译、静态分析为强证据。

技术上，PromptForge 站在 openJiuwen 生态（WorkSwarm/AgentOS、Agent Framework、Agent Studio、Agent System Service）与 Pangu/Qwen/DeepSeek 多模型之上，自研聚焦"需求编译、证据协议、评审去偏、MCDM 决策、预算调度"五个方法层模块；成本**算例演示（参数为假设值，非实测）**显示：3 候选 × 2 轮 × 4 评委约 ¥1.14/任务，是单模型单轮的 6.47 倍，但只是"单模型 self-refine 长跑"的 **2.16 倍**；经济档（剪枝/弱评委/缓存/早停）可压到单轮的 **3~5 倍**（约单模型长跑的 **1.2~1.7 倍【推断】**）——对高价值任务，多方案决策在经济上成立，实际数字须以真实报价与实测替换（见附录 G）。

在首届 openJiuwen 智能体创新大赛中，本作品对应 Coding 方向，按"创意（创新性/实用性/技术可行性/场景价值）→ DEMO（完成度/用户体验/技术难度/商业价值）"两阶段准备，并已按官方时间（10-31 报名、11-10 创意、12-27 DEMO）与开源合规要求（Apache-2.0、第三方代码 ≤30%、数据安全）完成倒排。

> **一句话**：PromptForge 是 AI 开发者的"方案编译器 + 多智能体评审会 + 机机联动执行网络"，让每个开发者都请得起一个由多个模型组成的专家评审团。

文档说明

- **本文档的用途**：① 团队内部设计与开发依据；② 官方"技术报告/项目文档"的底稿；③ 创意 PPT 的内容来源（逐页映射见附录 C）；④ 后续迭代的基线（变更须记录于附录 G.4 与修订记录）。
- **配套材料**：官方创意 PPT 模板（`_build/api/template_CN.pptx`）、官方 FAQ（`_build/api/FAQ.pdf`）、竞赛与调研笔记（`research/competition.md`、`research/tech.md`、`research/eval-cost.md`）、《PromptForge 实施工作台 v1.1》。

- **阅读路径**：产品/评审看第 1、2、4、6 章；开发看第 2、3 章与附录 G；比赛提交看第 1.3、7 章与附录 C/D/E；数据口径看「数据与信息规范」与附录 G。

数据与信息规范（v1.1 新增）

本文档严格区分五类信息，**不存在任何伪造的测试数据或效果数据**：

类别	标记	含义与使用规则
官方事实	**【事实】**	来自官方页面/FAQ/平台接口/官方文档，附来源与抓取日期；提交前复核
文献结论	引用链接	来自论文/官方文档的公开结论，保留原文口径与链接，不二次演绎
数学算例	**【算例】**	人为构造的参数与结果，仅用于演示公式与计算步骤，**不代表实测效果**；实际参数须替换并登记台账
设计目标	**【设计目标】**	产品验收目标，不是当前结果；须在 DEMO 阶段用 A/B 实测校准
推断	**【推断】**	基于事实与经验的分析判断，须显式标注，不得混入结论

- 任何效果数字（一次通过率、返工轮数、成本倍数等）在实测前只能表述为"设计目标/算例/推断"，不得写成"实测结果"。
- 实测数据统一记录在附录 G 数据台账与实验日志中，原始日志随仓库提交，禁止手工篡改或选择性引用。
- 竞赛信息以官方页面与官方 FAQ 为准（调研日期 2026-10-20；v1.1 复核日期 2026-10-23），时间口径不一致处一律取更早的保守值。

修订记录

版本	日期	主要变更
v1.0	2026-10-20	初稿：系统架构、核心机制、可行性、创新点、实施计划、答辩预演与附录
v1.1	2026-10-23	① 删除原稿中两类不适用内容（媒体输入误区、非官方渠道调用），多模态改为"关键帧 + 工具提取 + 结构化回读"的可选证据能力；② 新增「数据与信息规范」与附录 G「数据与假设台账」，逐项标注数据性质；③ 重排文档结构与样式，补充后续开发所需的阅读路径与工作台接口
v1.2	2026-10-30	① 新增 3.11「机机联动层」：规划机/执行机/评审机 + IAOV 协议 + 跨执行体层级，明确回应原始想法 2；② 新增 7.6「机机联动演进路线」；③ 摘要、定位、架构、多 Agent、5.6、6.1 同步补充；④ 创意 PPT 改为设计文档摘要版，删除未来计划页

目录

（页码基于 v1.2 排版；如修改内容请核对章节目录）

1. 设计目的（Why） **6**

1.1 背景：AI 开发的瓶颈已经从"模型能力"转向"需求表达与方案选择" 6

1.2 设计目的（可判定） 6

1.3 竞赛对齐：首届 openJiuwen 智能体创新大赛 6

1.4 目标用户与核心场景 9

1.5 范围与"不做的事" 9

2. 系统概述（What） **10**

2.1 一句话定位 10

2.2 总体架构 11

2.3 三类贯通状态 12

2.4 术语表 12

2.5 系统边界 13

2.6 两种产品形态 13

2.7 用户旅程总览 14

2.8 多 Agent 角色矩阵（对应官方"Agent 应用"要求） 14

3. 核心机制（How） **15**

3.1 需求编译（关键词 → SRD） 15

3.2 多候选提示词生成（Prompt Portfolio） 16

3.3 执行层：模型网关、沙箱与机器数据回读 17

3.4 评审聚合与偏差校正 20

3.5 多准则决策与对比可视化 22

3.6 融合与迭代 24

3.7 成本模型与停止准则 25

3.8 双形态架构：Skill 与 Orchestrator 的分工（解决"头脑与身体脱节"） 28

3.9 openJiuwen 生态使用说明与部署 29

3.10 安全、隐私与合规设计 31

3.11 机机联动层（Machine Link Layer） 32

4. 预期体验（So What） **35**

4.1 三类用户与前后对比 35

4.2 分阶段体验 35

4.3 三个典型场景 35

4.4 10 分钟现场 Demo 脚本（答辩用） 36

4.5 可观测指标与验证计划 36

4.6 设计风险与验证指标 37

5. 可行性评估与优化建议 **38**

5.1 总览结论 38

5.2 技术可行性（按模块） 38

5.3 工程可行性（比赛周期视角） 39

5.4 成本可行性.....40
5.5 合规、伦理与可持续性.....40
5.6 对用户四个原始想法的详细评审.....41
5.7 八条可执行优化建议.....42
5.8 竞赛可行性：按官方维度自评.....42
6. 创新点与差异化.....44
6.1 五个创新点.....44
6.2 与相关产品/方案的差异.....45
6.3 关键设计决策与技术依据对照.....45
6.4 与"多智能体辩论"的边界.....46
6.5 与 openJiuwen 生态的关系.....46
7. 实施计划与提交物.....48
7.1 团队角色与分工（1~5 人，建议 4~5 人）.....48
7.2 倒排里程碑（以 2026-10-20 为起点）.....48
7.3 提交物清单（创意阶段 / DEMO 阶段）.....49
7.4 开源与合规计划（获奖强制项，提前做）.....50
7.5 风险与缓冲.....50
7.6 机机联动的演进路线（设计目标，不是当前承诺）.....51
8. 答辩预演：评委可能问的问题.....52
8.1 高频问题与回答要点.....52
8.2 三类叙事版本.....54
附录 A：数学模型与算例摘要.....55
附录 B：技术依据与参考来源.....57
附录 C：官方创意 PPT 逐页大纲（创意提交版）.....60
附录 D：官方评审量规自评表（1–5 分）.....62
附录 E：提交检查清单.....63
附录 F：术语缩写与变更记录.....65
附录 G：数据与假设台账.....66
G.1 数据分级与引用规则.....66
G.2 数据与假设台账（v1.1 初始版）.....66
G.3 实验记录规范.....67
G.4 对外表述与变更规则.....67

1. 设计目的（Why）

1.1 背景：AI 开发的瓶颈已经从"模型能力"转向"需求表达与方案选择"

今天的大模型已经能写代码、做设计、生成内容，但开发者仍然要花大量时间在"来回改"上：一句话需求丢给 AI，得到一版"看起来对"的方案，跑起来才发现方向不对；再补充、再修改，几轮之后 token 和时间都花掉了，最初的方案目标反而模糊了。问题的根源不在模型不够强，而在三件事：

1. **需求没有被编译**：关键词/口语需求缺少约束、验收标准和边界，模型只能猜；不同开发者的表达水平差异，直接放大为方案质量方差。
2. **方案没有被比较**：单模型单次输出没有参照系；模型"自评"又存在位置、长度、自我偏好等偏差，开发者无法判断哪条路更优。
3. **结果没有被验证**：多数提示词工具只优化文本，不知道代码能不能跑、测试过不过、性能达不达标；"提示词看起来更好"与"方案真的更好"之间没有证据链。

对 AI 开发者而言，这意味着"一次做对"很难；对非专业开发者而言，这意味着"根本问不对"。PromptForge 想解决的，正是这三件事。

1.2 设计目的（可判定）

编号	目的	判定标准（目标值需实测校准）
P1	把关键词/口语需求编译成**含可执行验收标准**的方案规格	SRD 完整性 ≥ 0.8；验收标准 100% 可执行判定
P2	让多个**假设不同**的方案同题竞演，而不是单模型单次输出	每任务 ≥ 3 个候选，候选相似度 < 0.95，假设差异可解释
P3	用**机器可读证据 + 异构多评委**做可解释、带置信度的选优	推荐 Top-1 与人工评审一致率 ≥ 60%；评审 $\alpha \geq 0.6$
P4	让成本**可见、可控、可优化**，让多方案决策在经济上成立	单位任务成本 ≤ 同等质量单模型试错的 1.5 倍；预算命中率 ≥ 90%
P5	让非专家也能获得专家级提示词与方案（普惠）	新手用户首次任务可用率 ≥ 80%；澄清完成率 ≥ 80%

> 这五条目的对应关系：P1 解决"说不清"，P2/P3 解决"选不对"，P4 解决"用不起"，P5 解决"不会用"。

1.3 竞赛对齐：首届 openJiuwen 智能体创新大赛

本文档同时是参赛作品的技术设计文档。竞赛信息以官方页面与官方 FAQ 为准（调研日期 2026-10-20），以下为**已确认事实**与我们的对齐策略。

1.3.1 大赛基本盘（事实）

项	内容	来源
赛事名称	首届 **openJiuwen 智能体创新大赛**	官方赛事页 `1300000328/introduction`
主题	**「智体共生 · 创新无限」**	官方赛事页

项	内容	来源
定位	华为开发者大赛 ·应用创新实践赛；主办 openJiuwen 开源社区，承办华为技术有限公司	官方赛事情介绍页卡、官方 FAQ
奖金池	¥385,000（一等奖 3×3 万、二等奖 6×2 万、三等奖 15×1 万、入围奖 25×0.1 万）	平台接口字段与奖项页卡
参赛对象	全球开放，个人或团队（1~5 人，含队长）；每人只能属于一支队伍；每队只能提交 1 个作品	官方 FAQ Q3、Q8–Q13
报名费	免费	官方 FAQ Q5
赛事规模	截至 2026-10-20：513 人 / 185 支队伍报名	平台接口
评审方式	不刷榜、不设自动排行榜；创意评审 → DEMO 评审 → 决赛路演（30 支队伍）	官方 FAQ Q24/Q25/Q33/Q34

1.3.2 赛程与提交节点（官方口径存在多处不一致，按最保守时间倒排）

节点	时间	依据
平台报名/组队/初赛提交窗口	**2026-09-23 16:00 — 2026-10-31 15:59:59**（硬截止）	平台阶段配置接口
创意（初赛）提交截止	**2026-11-10**	官方页面「赛事介绍」页卡；官方 FAQ 附表 1
创意评审	2026-11-25 前完成	官方页面；官方 FAQ
DEMO 开发	2026-11-25 — **2026-12-27 24:00**（FAQ 正文另写 11-15 起，以页面 11-25 为准更稳）	官方页面 vs FAQ Q23，存在不一致
DEMO 评审	2026-12-28（社区专家线上）	官方页面；官方 FAQ
决赛	2027 年 1 月底（线上路演，30 支队伍）	官方 FAQ 附表 1

> **执行口径：10-31 前完成华为云账号实名认证 + 平台报名 + 组队；11-10 前提交创意 PPT；12-27 24:00 前提交 DEMO 四件套**。报名与提交阶段均需华为云账号实名认证；赛事平台与官方 FAQ 的时间不一致处，本文档一律取更早的保守值。

1.3.3 赛道选择：Coding

大赛设四个参考方向：办公、Coding、科研、设备互联（可选一个或多个，不强制单赛道）。**本作品选择 Coding 赛道**，理由：

1. 目标用户是 AI 开发者/独立开发者，Coding 是提示词质量痛点最尖锐、返工成本最可量化的场景；
2. 代码类任务的"证据"最硬（编译、测试、静态分析、性能），天然适合我们"证据驱动决策"的方法论；
3. 官方 Coding 方向明确包含**代码生成、代码审查、Bug 修复、自动化测试、项目脚手架、代码重构**，与本作品的"需求编译 → 多方案 → 评审 → 交付"链路直接吻合；

4. 同一套方法可低成本扩展到办公（文档/报表）与科研（文献/实验设计）方向，具备"一个作品、多场景延展"的叙事空间。
5. Coding 场景的 IDE、CI、浏览器是机机联动的第一批执行端点；设备/机器人作为方法层的延展，不在本次 Coding 交付范围内，但协议与状态对象保持一致（见 3.11、7.6）。

1.3.4 官方评审维度与我们的自评策略

阶段	官方维度（事实）	我们的回应	对应章节
创意评审	创新性、实用性、技术可行性、场景价值	证据驱动决策 + 去偏评审 + 成本编排（创新）；返工减少/门槛降低（实用与场景）；成熟模块 + 论文依据 + MVP 计划（可行性）	§4、§5、§6
官方模板附录量规（1–5 分）	项目运行、创新与创造力、对现实世界的影响、主题一致性	创意阶段即准备可运行原型/DEMO 截图；主题对齐"智体共生"（多智能体协同决策）；现实影响写"让每个开发者请得起 AI 专家评审团"	§4、附录 D
DEMO 评审	完成度、用户体验、技术难度、商业价值	可运行原型 + 10 分钟演示脚本 + 会议模式 + 成本账单 + 三层商业模式	§4.4、§8
决赛	线上路演	30 秒/3 分钟/电梯版叙事 + 现场 Demo 兜底	§8.2

1.3.5 官方要求与本文档的对齐清单

官方要求（事实）	我们的落实	位置
必须是围绕 Agent 的应用	PromptForge 本身是多 Agent 系统：需求编译 Agent、候选生成 Agent、执行 Agent、评审 Agent、决策/融合 Agent、成本管家 Agent	§2.2、§3
推荐（非强制）使用 openJiuwen WorkSwarm / AgentOS，使用生态在技术方案维度加分	明确组件级使用方案 + 自研边界（见 3.9「openJiuwen 生态使用说明」）	§3.9
支持 Pangu / Qwen / DeepSeek 等主流模型	多模型网关把三家模型作为候选执行器与异构评委	§3.3
算力使用华为云 AI 开发者空间	部署与演示环境方案	§3.9
创意提交：官方模板 PPT + zip ≤500MB + 每阶段 ≤5 次上传 + 命名规范	附录 C 给出逐页映射；提交检查清单见附录 E	附录 C/E
DEMO 提交：可运行原型 + 3 分钟视频 + 答辩 PPT + 技术报告	里程碑与提交物清单	§7
获奖须 Apache-2.0 开源、第三方代码 ≤30%、数据/安全合规	开源与合规计划前置到设计中	§5.5、§7.4
11-10 创意截止 / 12-27 DEMO 截止	倒排计划（以 10-20 为起点）	§7.2

1.4 目标用户与核心场景

用户	场景	核心诉求
独立开发者 / 学生	从一句话想法做出可运行原型	少走弯路、成本可控、能看到"为什么这个方案更好"
企业研发团队	技术方案选型、评审会、内部规范落地	可比较、可审计、可复现；沉淀团队知识
公民开发者 / 非专业用户	用自然语言完成小工具/自动化	系统帮我问对问题、补全验收标准
AI 平台/Agent 开发者	把"提示词优化"能力嵌入自己的产品	标准协议（MCP/A2A）+ 可计费 + 可观测

1.5 范围与"不做的事"

做：关键词→SRD 编译、多候选提示词、多模型执行、证据回读、多评委决策、融合迭代、成本账单、交付包；以及嵌入编码 Agent 的轻量 Skill 形态。

不做（v1.0）：不训练/微调基础模型；不替代 IDE；不做通用模型评测榜单；不做无限制自主 Agent；不做生产发布流水线接管；所有模型调用仅通过官方 API、云服务或本地部署。

本节小结：PromptForge 的设计目的可以压缩成一句话——**让 AI 开发从"反复试错"变成"一次选对"**；而首届 openJiuwen 智能体创新大赛的 Coding 赛道、Agent 应用定位与 openJiuwen 生态加分机制，恰好为这个目标提供了最合适的舞台。竞赛要求与设计内容的逐项对齐见 1.3.5，提交物与时间安排见第 7 章与附录 C/E。

2. 系统概述（What）

PromptForge 把"人对着一个大模型反复试错"改造成一条可度量、可复现的七阶段流水线：**需求编译 → 多候选提示词生成 → 多模型并行执行 → 机器可读证据采集 → 异构多评委评审 → 多准则决策与融合 → 迭代收敛与交付**。这条流水线由三类贯穿状态支撑：**需求规格 SRD**（我们到底要什么）、**证据包 Evidence Packet**（机器实际做成了什么）、**提示词版本 DAG**（每个方案是怎么演化来的）。系统提供两种形态：面向团队与跨模型的**外置编排器 Orchestrator**，以及嵌入 Codex / DSH / Claude Code 等编码 Agent 内部的**轻量 Skill**；两者共享同一套数据契约，可以单机跑，也可以混合跑。

2.1 一句话定位

> **PromptForge 是 AI 开发者的"提示词编译器 + 方案评审会 + 机机联动执行网络"**：输入关键词或一句话需求，输出一组**带执行证据、可横向比较、可解释**的高质量方案，并给出推荐项与次优备选。

类比：

传统软件工程	PromptForge 对应物
需求文档（PRD）	结构化需求规格 SRD（含可判定验收标准）
编译器 / 中间表示（IR）	提示词中间表示 PIR（目标、约束、上下文、验收、输出契约）
多套候选实现	多候选提示词 Portfolio（每个候选对应一组不同的架构假设）
单元测试 / CI	执行沙箱与证据采集（测试、lint、构建、性能、截图）
Code Review / 评审会	异构多评委 + 规则评审 + 方案对比看板
版本控制	提示词版本 DAG（每次变异/融合都有父节点与证据）
上线决策	多准则决策（TOPSIS / Pareto）与交付包

2.2 总体架构

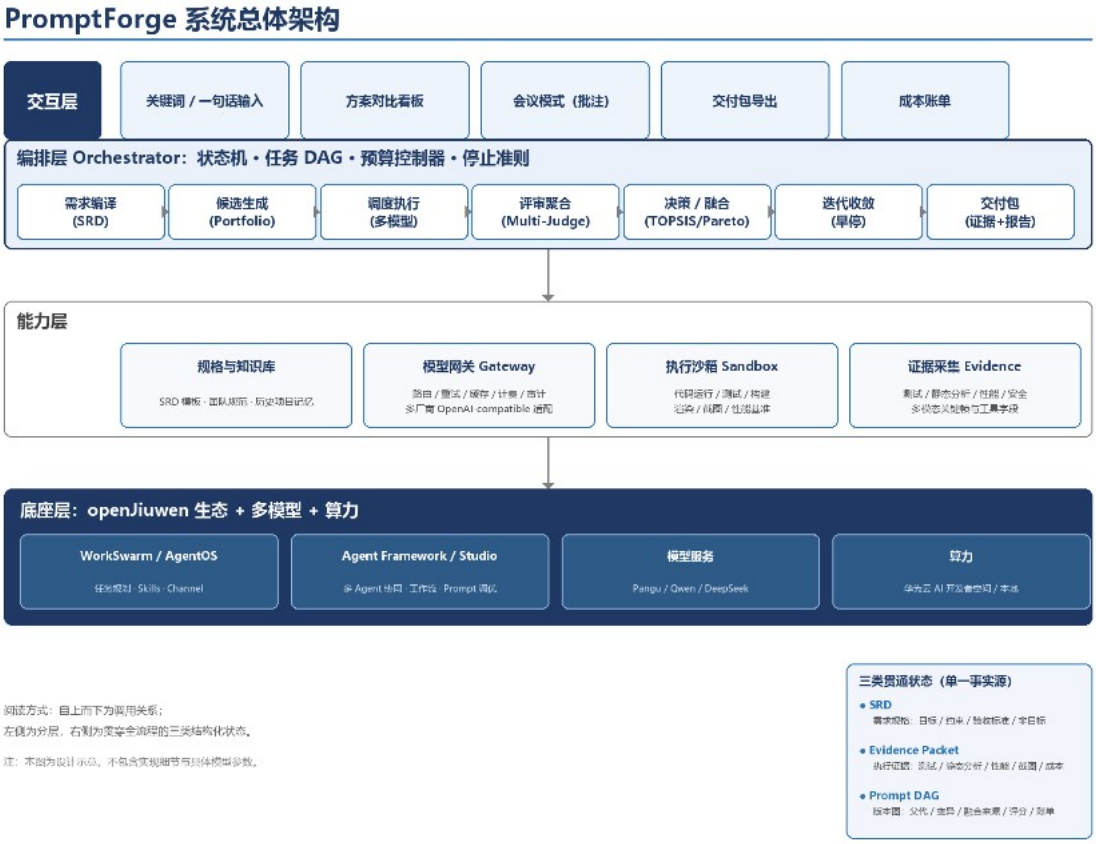


图 2-1 PromptForge 系统总体架构（设计示意）

分层职责

层	职责	关键组件	失败时如何降级
交互层	收集需求、展示对比与决策依据、人工批注	Web 工作台、CLI、IDE 插件、Skill	无前端时 CLI 可用
编排层	状态机推进、预算与停止控制、错误恢复	Orchestrator、任务 DAG、Token Ledger	单轮降级为"单候选 + 规则评审"
能力层	生成、执行、取证、评审	模型网关、沙箱、证据采集器、评委池	网关故障切换备用模型；沙箱故障转静态评审
底座层	Agent 运行时、模型服务、算力、存储与安全	openJiuwen WorkSwarm/AgentOS、Agent Framework、Agent Runtime、Pangu/Qwen/DeepSeek、华为云 AI 开发者空间	本地 Docker + 开源模型单机模式

> 设计原则：**编排层不直接"思考"，只做调度与仲裁**；所有判断必须落到"证据 + 规则 + 可复现的评分"上，避免变成又一个不可解释的 Agent 套壳。

在编排层与能力层之间，v1.2 增加了**机机联动总线（Machine Link Bus）**：它不替 Agent 思考，只把 `Intent / Action / Observation / Verdict` 四类消息路由给正确的执行体，并负责能力发现、

权限、预算、重试与审计。这样"执行结果回读"不再是一次性的日志采集，而是可复用的联动闭环；协议细节见 3.11。

2.3 三类贯通状态

状态对象	作用	生产者	消费者	生命周期
SRD（Structured Requirement Doc，结构化需求规格）	把模糊输入固化为可判定的目标、约束、验收标准	需求编译器 + 用户确认	候选生成器、评委、验收器	每次任务从 v0 开始，用户可编辑，变更留痕
Evidence Packet（证据包）	把"机器做出来的东西"转成可验证、可比较的结构化数据	执行沙箱 / 遥测采集器 / 多模态提取器	评委、决策器、可视化、审计	每个候选每次执行一份，按内容哈希寻址
Prompt DAG（提示词版本图）	记录候选的父代、变异说明、融合来源、评分与成本	编排器	迭代器、交付包、复盘分析	任务级持久化，可回放

这三类状态构成"单一事实源"：评委只能基于 SRD + Evidence 打分，编排器只能基于评分与预算做决策，从而把"头脑（LLM）"和"身体（执行模型）"的脱节压缩到协议层可检查的范围内（详见 3.8）。

2.4 术语表

术语	英文	含义
关键词编译	Requirement Compilation	把关键词/一句话需求转成含验收标准的 SRD 的过程
候选组合	Prompt Portfolio	一组假设不同、可并行执行的候选提示词（不是同义改写）
执行契约	Execution Contract	每个候选必须声明的输出格式、验收命令、禁止项
证据包	Evidence Packet	测试结果、静态分析、构建产物、性能数据、截图/关键帧特征等结构化集合
规则评审	Deterministic Judge	用测试、lint、schema 校验等确定性手段打分，不依赖 LLM
异构评委	Heterogeneous Judges	不同厂商/不同规模/不同提示模板的评审模型，用于抵消单模型偏差
一致性系数	Krippendorff's α / ICC	衡量多评委打分一致程度；低于阈值触发仲裁
贴近度	Closeness Coefficient	TOPSIS 中候选与理想解的相对距离
帕累托前沿	Pareto Frontier	不被任何其他候选全面占优的方案集合（质量-成本-时延等）
预算控制器	Token Ledger / Budget Controller	记录每次调用的 token 与费用，按预算分配并触发停止
双形态	Orchestrator + Skill	外置多模型编排器与内嵌单模型 Skill 共享同一数据契约

2.5 系统边界

包含：

- 关键词/一句话/仓库上下文的需求编译与澄清；
- 多候选提示词的生成、去重、多样性控制与执行契约校验；
- 多模型调用（商业 API、开源自托管、本地推理）的统一网关与路由；
- 沙箱执行、测试、静态分析、性能采集与多模态信息提取；
- 多评委评分、偏差校正、一致性检验、多准则决策、方案对比可视化；
- 优点抽取与融合、回归验证、迭代收敛；
- 交付包（SRD + 提示词 + 证据 + 评审报告 + 成本账单）导出与版本回放；
- 跨执行体（本机/远端/CI/浏览器）的机机联动编排、证据回传与审计（设计见 3.11）；
- 轻量 Skill 形态：单模型、单会话内的快速自迭代。

不包含（至少 v1.0 不做，避免范围失控）：

- 不训练/微调基础大模型（可作为二期用开源小模型蒸馏评审/路由模型）；
- 不替代 IDE、不直接接管生产发布流水线（只输出可审阅的交付物）；
- 不承诺"零人工"：高风险决策必须保留人工一键确认；
- 不做"无限制自主 Agent"（轮数、预算、权限均受控）；
- 不做"无协议、无权限、无预算"的开放式多机聊天；高风险物理动作必须人工确认；
- 不做模型能力排行或通用评测平台（只针对当前任务的方案比较）。

2.6 两种产品形态

维度	Orchestrator（外置编排器）	Skill（内嵌轻量形态）
运行位置	华为云/本地服务，跨会话	Codex / DSH / Claude Code 等 Agent 内
能力范围	多模型、多候选、多评委、可视化、版本回放	单模型内循环：改写提示词 + 自检 + 返工
典型场景	需求评审、方案选型、团队协作、交付	编码过程中"随手把提示词质量拉高"
成本量级	较高（预算可控）	低（约 1.2~2 倍单次调用，估算，见 3.7）
延迟	分钟级	秒级
数据契约	SRD + Evidence + DAG 全量	SRD-lite + 评分卡摘要
二者关系	下发"任务目标 + 验收标准 + 预算"	回传"执行证据 + 局部优化结果"

> **为什么必须双形态：**用户原始想法 3 的担忧是对的——如果所有判断都交给外部 LLM、执行端完全"无脑"，就会出现"头脑与身体脱节"：执行模型不知道全局目标，评审模型看不到执行细节，来回沟通把任务拖慢。解法不是二选一，而是**让执行端具备最小的自我修正能力（Skill 内循环），让外部编排器只负责跨模型、跨会话、带预算的决策（Orchestrator）**，中间用结构化协议对齐（详见 3.8）。

2.7 用户旅程总览



图 2-2 一次任务的七阶段流水线（设计示意）

2.8 多 Agent 角色矩阵（对应官方"Agent 应用"要求）

PromptForge 不是一个"套壳聊天框"，而是由 6 个职责清晰的 Agent 组成的协同系统；每个 Agent 都可以在 openJiuwen 的 WorkSwarm / Agent Framework 上以 ReActAgent 或 WorkflowAgent 形态实现，并通过 Agent System Service 的通信总线协作：

Agent	职责	输入/输出	主要能力	openJiuwen 映射
需求编译 Agent	澄清、槽位抽取、验收标准生成	关键词 → SRD	结构化输出、反问、领域模板	WorkflowAgent + Agent Studio 工作流
候选生成 Agent	假设矩阵采样、PIR 组装、去重	SRD → 3~5 候选	提示词生成、多样性控制、自我精炼	ReActAgent + Agent Studio Prompt 调优
执行 Agent	网关路由、沙箱执行、证据采集	候选 → Evidence Packet	工具调用、代码执行、结果结构化	Agent Framework 工具调用 + Agent System Service 沙箱
评审 Agent	盲评、换序、偏差校正、打分	SRD + Evidence → Score Card	多模型评委、评分一致性、置信区间	多 Agent 协同（异构模型）
决策/融合 Agent	TOPSIS/Pareto、优点抽取、回归验证	Score Card → 推荐 + 融合方案	多准则决策、集合覆盖、回归触发	JiuwenSwarm 协同 + Agent 自演进
成本管家 Agent	计量、预算分配、早停	全链路调用 → 账单 + 停止信号	Token Ledger、UCB 分配、预算告警	Agent Framework 调度 + 记忆子系统

> **设计要点**：Agent 之间不传"自然语言聊天记录"，只传 SRD / Evidence / ScoreCard 三类结构化对象；这样既避免上下文膨胀（呼应 WorkSwarm 的"上下文瘦身"能力），也让每一步可以独立评测与替换。

在六个 Agent 之外，v1.2 增加一条**机机联动总线**（不是第七个 Agent）：它把"谁在什么机器上、被允许做什么、实际观察到了什么、该如何判定"变成可路由、可审计的消息，解决多 Agent 跨进程/跨环境协作时的失联与黑箱问题（见 3.11）。

本节小结：PromptForge 的本质是给"提示词工程"补上软件工程的三件基础设施——可判定的需求（SRD）、可验证的证据（Evidence）、可回放的版本（DAG）；外置编排器负责"多方案决策"，内嵌 Skill 负责"单会话快速自迭代"，多 Agent 按结构化协议分工协作，整体可以运行在 openJiuwen WorkSwarm / AgentOS 之上。

3. 核心机制（How）

本章按"一次任务的真实流转顺序"展开：需求编译 → 候选生成 → 执行取证 → 评审聚合 → 决策与融合 → 成本控制与迭代停止 → 双形态协同 → openJiuwen 生态落地 → 安全合规。每个机制统一按**触发条件 → 处理规则 → 数据结构/公式 → 边界情况 → 与其他机制的耦合**描述。

阶段	输入	核心动作	输出	失败时
3.1 需求编译	关键词/一句话/仓库	槽位抽取、歧义检测、验收标准生成	SRD v1	主动澄清 ≤3 问；仍不明确则拒绝开工
3.2 候选生成	SRD	假设矩阵采样、去重、预评审	3~5 个候选提示词	全部不合格则回到 3.1
3.3 执行取证	候选 + 沙箱	多模型并行执行、测试、采集	Evidence Packet	网关降级/重试；执行失败记 0 分并记录原因
3.4 评审聚合	SRD + Evidence	异构盲评、偏差校正、一致性检验	结构化评分 + 置信区间	一致性低则加评委/人工仲裁
3.5 决策可视化	评分矩阵	TOPSIS/Pareto、对比看板	推荐方案 + 次优备选	硬约束全失败则扩大候选
3.6 融合迭代	方案优点	优点抽取、集合覆盖、回归验证	融合方案 vN	融合失败回退最优单方案
3.7 成本与停止	全链路计量	Token Ledger、预算调度、早停	成本账单 + 停止决策	超预算立即收口，输出当前最优

3.1 需求编译（关键词 → SRD）

触发条件：用户输入关键词、一句话、需求描述，或粘贴一个代码仓库作为上下文。

处理规则（5 步）

1. **意图分类**：新建系统 / 修改现有代码 / 方案咨询 / 代码审查 / 多媒体内容生成，不同意图走不同模板。
2. **槽位抽取**：目标（Goal）、交付物（Deliverables）、技术栈（Stack）、约束（Constraints）、验收标准（Acceptance Criteria）、非目标（Non-goals）、风险（Risks）。
3. **歧义检测**：对每个关键槽位计算"缺失度 × 影响权重"，生成待澄清问题并按"信息增益"排序。
4. **验收标准生成**：把"做好看/要流畅"这类不可判定表述翻译成可执行的代理指标（如"首屏 LCP < 2.5s""通过 12 条 Playwright 用例""p95 延迟 < 200ms"）。
5. **反向确认**：用 5 行话回显"我理解的任务 + 验收标准 + 我替你猜的部分"，用户一键确认或修改。

SRD Schema（核心字段）

字段	必填	示例	校验规则
`goal`	是	"为校园社团做一个活动报名 Web 应用"	≤ 80 字，含动词与对象
`deliverables`	是	`["可运行仓库", "部署脚本", "README"]`	至少 1 项，可验证

字段	必填	示例	校验规则
stack	否	["Next.js", "SQLite"]	不在团队规范内则提示
constraints	否	"单机可跑、无外部付费依赖"	与预算/合规规则冲突检测
acceptance	是	12 条可执行用例 + 3 条性能阈值	每条必须能"跑出通过/失败"
non_goals	是	"不做支付、不做多租户"	至少 1 条，防止范围膨胀
ambiguity_score	自动	0~1	> 0.2 触发澄清
open_questions	自动	[{"q":"用户规模?","impact":"高"}]	≤ 3 条，带默认建议

公式：完整性评分与澄清阈值

设必填槽位集合为 S，第 i 个槽位的完成度为 $c_i \in [0,1]$，影响权重为 w_i（$\Sigma w_i = 1$），则

$$Q_{srd} = \Sigma_{i \in S} w_i c_i, \quad \text{若 } Q_{srd} < 0.8 \text{ 则触发澄清}$$

澄清问题数量受控：$n_{ask} = \min(3, \lceil (0.8 - Q_{srd}) \times 10 \rceil)$，每题必须提供默认选项，避免"把思考成本转嫁给用户"。

边界情况

- 需求涉及用户无法描述的私有系统：允许上传仓库/接口文档作为上下文，SRD 中标注"基于代码推断"。
- 目标本身不可验证（"有高级感"）：必须转化为代理指标，并在 SRD 中注明"代理指标 ≠ 原目标"的风险。
- 用户拒绝澄清：按默认假设继续，但 SRD 中把假设标红，并在交付包中列出"因假设导致的返工风险"。

与其他机制的耦合：SRD 是 3.4 评审 Rubric 的锚点、3.6 融合冲突的裁判依据；SRD 一旦变更，已有候选标记为 `stale` 并触发重生成（可增量复用证据）。

本节小结：需求编译的目标不是"写得详细"，而是把需求变成**可判定的契约**；这一步做不好，后面所有评分都是在错误的目标上算分。

3.2 多候选提示词生成（Prompt Portfolio）

触发条件：SRD 确认完成，且 `ambiguity_score ≤ 0.2`。

核心原则：候选之间是"假设不同"，不是"措辞不同"。 同义改写的候选会让评审失去信息量，只是花钱买噪声。每个候选必须显式声明一组假设，例如：

假设维度	候选 A	候选 B	候选 C
架构风格	单体优先，最快跑通	前后端分离，便于扩展	Serverless，零运维
优化目标	功能正确性	可维护性/可测试性	性能/成本
实现策略	复用成熟模板	手写核心逻辑	引入轻量框架
验证策略	单元测试为主	E2E 为主	压测 + 安全扫描

假设维度	候选 A	候选 B	候选 C
风险偏好	保守，少依赖	均衡	激进，追新

生成流程

1. **假设矩阵采样**：从 6 个假设维度中各取 1 种，用拉丁超立方/贪心覆盖挑选 N=35 组差异最大的组合。
2. **PIR 组装**（Prompt Intermediate Representation）：每组假设填充为结构化提示词，固定 8 个区段——角色、目标、上下文、约束、验收标准、输出契约、示例、禁止项。
3. **多样性控制**：计算候选 embedding 两两余弦相似度；＞0.95 合并，＜0.85 视为有效差异；若有效差异不足，补充"反事实候选"（故意采用相反权衡）。
4. **预评审门禁**：用便宜的小模型按 4 项自检（能否直接执行、验收是否明确、边界是否覆盖、成本是否可控），低于阈值的候选先做 1 轮 self-refine 再进入执行，避免用昂贵的执行与评审成本去筛掉"提示词本身就烂"的候选。
5. **成本上限**：候选生成 token 控制在总预算的 10%~15%；N 的上限由预算与 3.7 的边际收益模型共同决定。

候选质量门禁表

检查项	权重	通过标准	失败处理
可执行性	0.35	输出契约明确到文件/命令级	补全输出契约
验收明确性	0.30	每条验收可跑出通过/失败	重写验收标准
边界覆盖	0.20	至少覆盖 3 类异常输入	补边界用例
成本可控	0.15	预估 token 在预算内	压缩上下文/换小模型

边界情况：所有候选预评审均不通过 → 回到 3.1，说明 SRD 本身不可执行；若因模型能力不足，则降低候选野心（例如先做 MVP 再做扩展），而不是继续堆提示词。

与其他机制的耦合：候选是 Prompt DAG 的根节点；候选的假设标签会进入 3.6 的优点抽取，因为"架构风格"通常不可融合，而"测试用例、边界处理、提示词片段"可融合。

本节小结：多候选的价值来自"假设多样性"；没有差异的候选只是把同一个错误答案买了很多遍。

3.3 执行层：模型网关、沙箱与机器数据回读

3.3.1 模型网关（Gateway）

定位：统一"调用谁、怎么调、花多少、记什么账"，是用户原始想法 1 中"中转站"的工程化版本——但不是简单转发，而是**路由 + 计费 + 审计 + 降级**。

接口：对外统一 OpenAI-compatible 协议（`/v1/chat/completions`、`/v1/embeddings`、结构化输出），对内适配不同厂商与本地推理（Pangu/Qwen/DeepSeek 云端 API、vLLM 本地推理、OpenAI/Anthropic 兼容接口）。

路由策略（按优先级）

路由	规则	目的
能力路由	按任务类型查模型能力表（代码/长文/多模态/推理）	用对的模型
成本路由	在满足能力阈值的模型池里选单位质量成本最低	省钱
延迟路由	交互态走低延迟小模型，后台批处理走大模型	体验
隐私路由	涉密/私有代码强制本地或专属实例	合规
可用性路由	主模型超时/限流自动切换备用模型，并记录降级原因	稳定

必做工程细节：幂等键防重复扣费、重试与指数退避、限流、Prompt Cache 命中统计、每次调用记录（模型、token、单价、耗时、trace_id）、失败分类（网络/限流/内容安全/格式错误）。

3.3.2 执行沙箱（Sandbox）

原则：把模型产出的代码当作**不可信代码**执行。

- 隔离：容器/微虚机，默认禁网（按需白名单），只读挂载基础镜像，工作目录可写但有配额。
- 资源：CPU/内存/磁盘/GPU 配额，墙钟超时（如 10 分钟），进程数限制。
- 标准化动作：`install → build → test → lint → type-check → benchmark → render/screenshot`，每步产出结构化结果。
- 可复现：记录镜像哈希、依赖锁文件、随机种子、环境变量（脱敏）。
- 安全：依赖安装走内网镜像；禁止 sudo、禁止访问宿主机路径；执行日志脱敏后入库。

3.3.3 证据包（Evidence Packet）

定义：每个候选每次执行产出一份 JSON 证据包，是评委唯一被允许使用的"事实输入"（评委可以看代码 diff，但不能凭空想象运行结果）。

证据组	字段示例	强度
可执行验证	`tests.passed / total / coverage、build.status、typecheck.errors`	强
静态质量	`lint.errors/warnings、complexity、duplication、sast.findings`	中强
性能	`latency.p50/p95、memory.peak、bundle.size`	中
安全	`dependency_vulns、secret_scan、permission_scope`	中强
多模态	`screenshots[]、video.keyframes[]` 的描述/OCR/ASR、ui_diff	中
差异	`diff.files/lines/hunks、churn`	中

证据组	字段示例	强度
成本	`tokens.input/output`、`usd.estimate`、`wall_time`	强
可复现	`image_hash`、`lockfile_hash`、`seed`	强

证据冲突处理：不同证据组给出矛盾信号时（例如"测试通过但安全扫描高危"），按硬约束优先：安全/合规失败直接一票否决；测试失败进入观察区；性能不达标按权重扣分。

3.3.4 多模态与工具证据（可选能力）

对于 UI、图表、演示视频、扫描文档等"非代码"产物，系统提供**可选**的多模态证据通道；该通道默认关闭，仅在任务确实涉及视觉/媒体内容时启用：

层级	做法	适用场景	成本口径
关键帧/截图	抽取关键帧或界面截图，交多模态模型理解	UI 还原、图表解读、演示内容摘要	按供应商官方 token 计费口径；参考量级：OpenAI 高细节 1024² 图片约 765 token、Claude 同尺寸约 1.4k token（官方文档口径，实际以当时文档为准）
工具提取	ffprobe 读元数据、OCR 读字幕、ASR 转写、CV 检测、UI 自动化截图对比	需要结构化描述的视频/图像	本地工具运行成本，可忽略 token
结构化回读	工具结果写入 Evidence Packet（`screenshots[]`、`ui_diff`、`ocr_text`、`asr_text`、`media_meta`）	评委据此评审	与代码证据同流程

使用原则

- 多模态证据属于"中证据"，用于 UI/体验/内容类主观维度；代码类任务仍以测试、编译、静态分析为强证据。
- 只把"必要关键帧 + 工具提取的结构化摘要"送入模型，控制上下文长度与成本；默认抽 6~10 张关键帧（按官方 token 口径估算约 0.5 万~1.4 万 token，实际以分辨率、帧数与供应商口径为准）。
- 视频类任务默认组合"关键帧 + ASR 字幕 + 元数据"；复杂视频应拆解为可验证子任务（截图对比、字幕校对、操作步骤校验）。
- 涉及隐私/敏感内容时先脱敏再送模型；默认走本地工具，必要时使用本地多模态模型。

边界：多模态评审不承诺"理解任意视频"；它只补充视觉/内容维度的证据，不能替代可执行验证。是否启用由任务分类器与用户配置共同决定。

本节小结：执行层是 PromptForge 与"纯提示词工具"的分水岭——只有真正把代码跑起来、把结果结构化，评审才有事实依据；对图像/视频类产物，用"关键帧 + 工具提取 + 结构化回读"补充证据，作为可选能力按需启用。

3.4 评审聚合与偏差校正

> **数据说明**：3.4–3.7 中的数字均为【算例】（人为构造的参数与结果，仅用于演示公式与计算步骤），不代表任何实测效果。落地时所有参数须以实际测量替换，并登记在附录 G 的数据台账中。

3.4.1 评审协议（先定协议，再谈数学）

- **评分制式**：日常用 1–5 分绝对分 rubric（成本低、可跨候选比较）；当 Top-2 差距小于噪声时，切换为**成对比较 + 位置互换**（更可靠，但比较次数随候选数平方增长）。
- **Rubric 固定**：评审前确定评分维度与锚点，评审中不允许改 rubric；每条评分必须"先引用证据，再给分"。
- **盲评**：隐藏候选来源与生成模型；候选呈现顺序随机化；成对比较做 A/B 与 B/A 两遍，用于估计位置偏差。
- **异构评委**：K=4（至少覆盖 2 个模型家族），评委尽量与候选生成模型不同家族；**任何评委不得评审自己参与生成的候选**。
- **机器可读输出**：`{"candidate_id", "criterion", "score", "evidence_refs", "rationale", "confidence"}`，方便聚合与审计。

> 【算例估算】在最常见的 3 候选 ×2 轮 ×4 评委配置下（参数见 3.7.3 假设表），成对比较（16 次）与绝对评分（20 次）成本几乎相同（+2.9%），只有做位置互换才翻倍；因此"单轮单顺序的成对比较"是控制尺度漂移最便宜的手段，位置互换只对差距 < 0.2 分的近邻对启用。该比例随单价与 token 画像变化，实测前不作为结论。

3.4.2 标准化与去极值（不能省的一步）

评委内标准化（消除不同评委的打分尺度差异）：

$$z_{kj}=(s_{kj}-\mu_k)/(\sigma_k),\quad s_{ij}=(\Sigma_k\ _k z_{kj})/(\Sigma_k\ _k)$$

K=4 时先去一个最高分、一个最低分取中间两个的均值（trimmed mean）；K≥5 用中位数或 MAD 剔除离群值。**直接把原始分平均，等于让打分尺度不同的评委按噪声加权。**

算例（3 候选 ×4 评委，0–10 分）

候选	J1	J2	J3	J4	原始均值	去极值均值	排序
A	8.0	8.5	8.4	8.2	8.275	8.300	2
B	9.0	9.2	**5.0**	8.8	8.000	**8.900**	**1**
C	7.5	7.8	7.6	7.7	7.650	7.650	3

J3 给 B 的 5.0 是离群低分（J3 对 A/C 打分正常），一个极端值让 B 从第 1 掉到第 2——**排序反转换来的是"多花一份执行成本去优化一个错误结论"**，所以去极值必须先做。

3.4.3 Bradley-Terry / Elo 聚合

Bradley-Terry（离线批量、多评委、多轮）：候选 i 战胜 j 的概率

$$P(i\ j)=(\theta_i)/(\theta_i+\theta_j)$$

用 MM 迭代求解（数值稳定、无需梯度）：θ_i arrow W_i / Σ_j i n_ij/(θ_i+θ_j)，再归一化，直到收敛。**算例**：4 候选、每对比较 8 次，MM 收敛得 θ=(A:0.495, C:0.213, B:0.186, D:0.106)。注意 B 与 C 胜场只差 1，P(C B)=0.534——**这类"弱排序"不能当结论**，应作"两者进入下一轮"的依据。

Elo（在线增量、多轮）：E_ij=1/(1+10^(R_j-R_i)/400)，R_iarrow R_i+K(S_ij-E_ij)；多评委取票面平均，多轮用 K_t=K_0/(1+t/τ) 递减。离线报告用 BT，实时看板用 Elo，二者都要用 bootstrap 给排序置信区间。

3.4.4 一致性检验与低一致性处理

指标	可用	勉强可用	不可用
Krippendorff's α	≥ 0.80	0.667–0.80（探索性）	< 0.667（< 0.5 必须停工）
Cohen/Fleiss κ	≥ 0.61	0.41–0.60	≤ 0.40
ICC(2,k)	≥ 0.75	0.50–0.75	< 0.50

每个指标分别算一致性（质量指标可能一致性高、可维护性可能低），并给 bootstrap 置信区间。低一致性时按根因处理：

根因	诊断	处理
rubric 模糊	分项一致性都低、理由不引用同一标准	拆子项 + 给 3 个锚点示例，重跑受影响评分
评委尺度不同	评委均值/方差差异大	评委内 z 标准化，或改秩/成对比较
单个离群评委	leave-one-out α 明显上升	降权或剔除，记录并复核该评委 prompt/模型
指标本身主观	硬指标一致、软指标不一致	软指标降权，主决策只用高一致性指标
系统偏差	位置互换分差大、长度与得分相关	先做 3.4.5 校正，再算一致性
仍 < 0.5	所有手段无效	停止采信该指标；只用可执行验证，文档明示不确定性

3.4.5 偏差校正

- **位置偏差**：同一对候选按 A/B 与 B/A 各评一次，b_pos=mean(s_AB-s_BA)，校正分取 (s_AB+s_BA)/2；|b_pos|>0.3（0–10 分制）的评委降低成对结论权重。
- **自我偏好**：跨模型家族评委 + 匿名化 + 评委不评自己生成的候选；评委家族在评审池占比 ≤ 1/3。
- **长度偏差**：s=β_0+β_1(len)+，用残差 作为去长度后的质量分；或把长度差作为 BT 协变量：logitP(i j)=θ_i-θ_j+β^→p(x_i-x_j)。
- **评审放大风险**：反复用评委分数选优会"先升后降"（reward overoptimization），因此**硬验证信号优先 + 最多 3 轮 + 早停**，不追求"把评委分刷到最高"。

本节小结：评审层的可靠性不是靠"多叫几个模型"，而是靠协议（盲评、异构、换序）、统计（标准化、去极值、一致性、置信区间）与硬证据三重保障；三者缺一，多评委就会变成"多花钱买噪声"。

3.5 多准则决策与对比可视化

3.5.1 指标体系、归一化与权重

两层指标树（一级 4 个）：

一级指标	权重（均衡预设）	二级指标	测量口径
质量 Q	0.45	正确性 0.50 / 完整性 0.25 / 可维护性 0.15 / 安全性 0.10	可执行验证通过率 + rubric；安全性为一票否决型
效率 E	0.15	时延 0.40 / Token 消耗 0.60	p50/p95 端到端秒数；每任务 input+output token
成本 C	0.20	单位任务成本	¥/任务（执行 + 评审 + 验证）
稳定性 S	0.20	稳定性	1-σ/μ（重复运行），失败率打折

三套预设权重（可配置）：A 均衡 `(0.30, 0.20, 0.20, 0.15, 0.15)`（正确性/完整性/成本/时延/稳定性，用于演示算例）；B 生产 7 维多级；C 成本敏感 `(0.25, 0.15, 0.35, 0.15, 0.10)`。

归一化：效益型 y=(x-x_min)/(x_max-x_min)；成本型反向；目标型/区间型按 SLO 衰减；重尾先 (1+x) 或秩归一。**基准必须固定**（需求基线/SLO/历史最优），不能取"本批候选的 min/max"，否则候选整体变差时仍会产出 1.0 分，且跨轮不可比。

权重确定：AHP 主观权重（CR<0.1）+ 熵权法客观权重 + 组合 w=α w^subj+(1-α)w^obj（α=0.6）。**敏感性**：对每个权重 ±0.05 扰动重算排序，Kendall τ ≥ 0.8 才采纳；τ 低说明结论对权重敏感，必须回到需求方协商，不能默默取平均。

3.5.2 TOPSIS：步骤与算例

```
Step 1 向量归一化 r_ij = x_ij / sqrt(Σ_k x_kj²)
Step 2 加权矩阵 v_ij = w_j · r_ij
Step 3 正/负理想解（效益型取最大/最小，成本型取最小/最大）
Step 4 欧氏距离 D_i⁺、D_i⁻
Step 5 贴近度 C_i = D_i⁻ / (D_i⁺ + D_i⁻)
Step 6 C_i 越大越优；并列比较 D_i⁺
```

算例（4 候选 × 5 指标，权重取预设 A）

候选	正确性	完整性	成本(¥)	时延(s)	稳定性	贴近度 C*	排名
A	82	76	38	42	0.88	0.5230	3
B	90	68	52	35	0.80	0.4008	4
C	74	88	26	55	0.92	**0.5642**	**1**
D	86	80	45	30	0.76	0.5253	2

解读（比结果更重要）：

1. C 胜在"成本最低 + 完整性/稳定性最高"，但**正确性垫底（74）**——若正确性有硬门槛（如 ≥80），C 应在排序前被硬约束过滤，TOPSIS 只是排序器，不负责守门；

2. A 与 D 的贴近度只差 0.0024（0.46%），小于评委噪声量级，**不能宣布 A 优于 D**；应做 bootstrap 秩稳定性检验，不稳定则并列进入下一轮；
3. 排序结论必须附"硬约束过滤 → Pareto 剪枝 → 排序 → 稳定性检验"的完整链条，而不是只报一个 TOPSIS 分数。

3.5.3 Pareto 前沿与超体积（HV）

支配关系：a 支配 b 当且仅当所有目标上 a 不差于 b 且至少一项严格更好；非支配集合即 Pareto 前沿。**作用**：权重未定时先剔除严格更差的候选，零信息损失。

超体积（成本最小化、质量最大化，参考点 r=(c_ref,q_ref)，前沿按成本升序）：

$$HV=\Sigma_{i=1}^{L}(c_{i+1}-c_i)(q_i-q_{ref})$$

算例：6 个候选（成本，质量），非支配集 {E(30,0.80), G(40,0.84), F(45,0.86), H(55,0.88)}，参考点 (60,0.75)，HV=2.70；新增候选 K(42,0.87) 支配 F 后 HV'=2.89，**ΔHV=+7.04%**。

HV 的三个用途：① 每轮只把前沿候选送入下一轮；② ΔHV < 1% 作为"再加候选/再加轮"边际贡献低的停止辅助信号；③ 对外报告"预算–质量前沿"比只报冠军更诚实。

3.5.4 加权和（SAW）vs TOPSIS

- **SAW**：$S_i=\Sigma_j w_j y_{ij}$，线性补偿、透明、与期望效用一致；缺点是一项极差可被其他高分补回。
- **TOPSIS**：与正/负理想解的相对距离，对"单项崩盘"惩罚更重，但结果依赖候选集（增删候选可能秩反转）。

反转示例（等权 3 指标）：X=(10,10,1)，Y=(6,6,6)；SAW 归一化后 X=0.667>Y=0.333，而 TOPSIS 下 X 因第三项远离正理想解，C_X=0.371<C_Y=0.629——**方法选择本质是价值取向，不能挑一个好看的结果**。

默认组合拳（直接实现）：硬约束过滤 → Pareto 前沿压到 2–3 个 → TOPSIS 定序 → bootstrap 秩稳定性 → Top-1 不稳定则前二并列进下一轮；SAW 全程作为交叉校验同步计算，两者 Kendall τ 写入日志。

3.5.5 对比可视化与"会议模式"

视图	内容	回答的问题
雷达图	各候选在 6~8 个指标上的归一化得分	谁强在哪、短板在哪
成本–质量散点（带 CI）	横轴总成本、纵轴质量分，误差棒 + Pareto 前沿	多花的钱值不值
评委–候选热力图	每个评委对每个候选的 z 分	谁在分歧、分歧多大
桑基图	生成/执行/评审/融合/回归的成本流向	钱花在哪
并排 diff	候选提示词与产出代码的差异	方案到底差在哪
争议点标注	一致性低或 CI 重叠的维度高亮	需要人工讨论哪里
决策卡（Decision Card）	推荐项 + 理由 + 次优备选 + 成本 + 置信度 + 未决清单	一页看懂结论

"会议模式"把上述视图投屏，允许成员对具体维度批注、锁定假设、指定融合项；系统把会议决议写回 Prompt DAG，形成"讨论—决策—留痕"的闭环。

本节小结：决策层的目标不是"算出一个冠军"，而是**让选择过程可解释、可复现、可辩论**：硬约束守门、Pareto 去噪、TOPSIS 定序、统计检验防误判、可视化让团队把分歧说清楚。

3.6 融合与迭代

3.6.1 把"优点"原子化

融合的本质不是"把两份提示词拼起来"，而是抽取**可合并的优点原子（atom）**再组合：

$$\text{atom_k}=(\text{id, 描述, 来源候选, 收益 } u_k, \text{成本 } c_k, \text{证据, 依赖/冲突集合})$$

- u_k：预期质量提升（0–1 质量点，优先用历史 A/B 效应量；无证据的条目打 0.5 折）；
- c_k：合并成本（复杂度/预算单位或 token）；
- 抽取流程：LLM 读评审理由 + 通过/失败用例 → 输出 atom JSON → 规则抽检 → 语义去重（相似度 > 0.85 合并）。

3.6.2 融合建模：集合覆盖 + 预算背包

加权集合覆盖（用最少成本覆盖所有必需优点 R）：

$$\min \Sigma_k c_k y_k \quad \text{s.t. } \Sigma_{k:e\in A_k} y_k \geq 1, \forall e\in R;\ y_k\in\{0,1\}$$

NP-hard，贪心按"新增覆盖收益/成本"取最大，近似比 1-1/e；条目少时直接 ILP 精确解。**算例**：必需覆盖 {e1 JSON, e4 自检, e5 引用, e7 拒答}，贪心选 P3(覆盖 e1/e4/e7, 成本 0.9) + P4(覆盖 e5, 成本 0.6) = **1.5**，且穷举验证为最优。

冲突感知 0–1 背包（预算内收益最大）：

$$\max \Sigma_k u_k y_k \quad \text{s.t. } \Sigma_k c_k y_k \leq B;\ y_k+y_l\leq 1, \forall (k,l)\in \text{Conflict};\ y_k \leq \Sigma_{l\in dep(k)} y_l$$

算例：8 条优点、预算 B=8，无冲突最优 {e1,e2,e3,e6,e7}，收益 35、成本 8；若 e1（纯 JSON 输出）与 e4（展示思维链自检）互斥且 e4 为硬需求，最优变为 {e2,e3,e4,e7}，收益降为 **30**（相对全局最优损失 5；若只是"e4 必选但 e1 可共存"，最优 {e1,e2,e4,e7} 收益 32，损失 2）——**冲突的代价必须显式报告给决策者**，不能偷偷丢掉一个优点。

3.6.3 冲突检测与消解

冲突类型	例子	处理
输出契约	"只输出 JSON" vs "展示推理"	拆两阶段调用/条件分支输出
指令矛盾	"必须先反问" vs "禁止反问"	按优先级取一，或改条件指令
长度/预算	"详细展开" vs "≤200 字"	上限优先，或按任务类型切模板
安全策略	越狱式角色扮演 vs 拒答策略	P0 硬约束，直接剪掉
重复冗余	两条都要求"标注不确定性"	语义去重保留收益大者
顺序依赖	先自检再输出 vs 单次生成	依赖图拓扑排序，无法满足则降级

流水线：规则过滤（快）→ 语义相似度/NLI（中）→ LLM 判别给证据（慢，仅高风险对）→ 人工复核（安全/合规类）；输出冲突图，按最大权重独立集选可并存条目，再进背包。

3.6.4 融合后强制回归验证

回归集：固定 golden set（30~100 条，覆盖主任务 + 边界 + 安全用例），融合版与每个原候选做配对比较。

- **McNemar 检验**：b=融合通过且基线失败，c=融合失败且基线通过，^2=(|b-c|-1)^2/(b+c)（df=1，临界 3.84）；小样本用精确二项检验。
- **算例**（30 条）：情形 1，b=6、c=3 → ^2=0.444<3.84，p=0.254，**不显著，不能宣称融合更好**；情形 2，b=8、c=1 → ^2=4.0>3.84，p=0.0195，**显著，可采纳**。
- **三条门同时满足才发布**：① 质量非劣（非劣性边界 ε=2 个百分点，bootstrap CI）；② 成本 ≤ 单任务预算；③ 安全/格式零回归（任何一条安全用例回归直接拒绝）。
- **回滚**：保留候选版与融合版双份 prompt，线上影子流量出现回归可一键回滚，日志记录融合版本号与 atom 来源。

3.6.5 迭代、变异与版本 DAG

- **变异**：针对评审指出的短板改条件/补边界/换实现假设（每次只改 1~2 个变量，保证可归因）；
- **交叉**：把两个候选的可融合优点组合成新候选；
- **剪枝**：每轮只保留 Pareto 前沿 + 多样性代表，N_t+1=max(N_min,⌈ρ N_t⌉)，ρ=2/3；
- **版本 DAG**：每个节点记录父节点、变异说明、证据包、评分、成本；支持任意历史版本回放与交付包重建；
- **上限**：默认最多 3 轮，收益递减或预算不足即停（见 3.7.5）。

本节小结：融合是"高风险、高回报"的可选动作：只有结构化优点、显式冲突代价与严格回归检验三者齐备时才做；否则老老实实交付最优单方案，反而是更专业的选择。

3.7 成本模型与停止准则

3.7.1 单轮成本公式

设生成/执行/评委模型单价分别为 (u^g_in,u^g_out)、(u^e_in,u^e_out)、(u^r_in,u^r_out)（元/token），token 画像分别为 T^g_in,T^g_out、T^e_in,T^e_out、T^r_in,T^r_out，任务数 m，候选数 N_t，评委数 K_t，可执行验证成本 C_verify：

$$C_{round}(t)=N_t(u^g_{in}T^g_{in}+u^g_{out}T^g_{out})\text{（候选生成）}+N_t\,m(u^e_{in}T^e_{in}+u^e_{out}T^e_{out})\text{（执行）}+N_t\,K_t(u^r_{in}T^r_{in}+u^r_{out}T^r_{out})\text{（评审）}+C_{verify}(N_t)$$

3.7.2 多轮递推与缓存

$$C_{total}=\Sigma_{t=1}^{T}C_{round}(t)+C_{fuse}+C_{reg},\quad N_{t+1}=\max(N_{min},\lceil\rho N_t\rceil)$$

带缓存（命中率 h，缓存价比例 ）的有效成本：C_eff(x)=x· u·[1-h(1-)]；批处理折扣：C_eff=C(1-γ_batch)。

3.7.3 成本算例演示（参数为假设值，非实测）

> 【算例】 以下单价与 token 画像为演示参数，仅用于说明计算过程，不代表真实报价或实测消耗；实际值须以供应商报价与实测 token 替换（登记于附录 G 数据台账）。

【示例假设】 强模型 ¥2/¥8 每百万 token（输入/输出），弱模型 ¥0.5/¥1.5；生成 800/1200 token、执行 2500/1500、评分 2200/150、成对比较 3200/100；m=10，N_1=3，ρ=2/3，K=4，T_max=3。价格量级参照 2026-10-20 DeepSeek 官方价（flash：缓存命中输入 0.02–0.04 元/M、未命中 1–2 元/M、输出 4–8 元/M，空闲半价；v4-pro：0.15–0.30 / 4.5–9.0 / 13.5–27 元/M），仅为量级对照，落地时按实际报价替换。

基线（单模型、单轮、单候选、1 评委）：执行 ¥0.1700 + 评审 ¥0.0056 = **¥0.1756/任务**。

多方案（2 轮 + 融合 + 回归）：

组成	成本	占比
轮 1（N=3, K=4）：生成 0.0336 + 执行 0.5100 + 评审 0.0672	¥0.6108	53.8%
轮 2（N=2, K=4）：生成 0.0224 + 执行 0.3400 + 评审 0.0448	¥0.4072	35.8%
融合（1 次 3000/2000 token）	¥0.0220	1.9%
回归验证（5 任务 × 执行 + 2 评委）	¥0.0962	8.5%
合计	**¥1.1362**	100%

成本倍数：相对单模型单轮 **6.47×**；**但相对"单模型 self-refine 跑 3 轮"（¥0.5268）只有 2.16×**——这是对用户"多方案会不会贵到接近集群模式"担忧的正面回答：**贵的不是多评委，而是"每个候选都要把全部任务跑一遍"**。

结构洞察：轮内执行占 83.5%，评审仅 11.0%，生成 5.5%。所以优化优先级是：

优化项	效果	优先级
候选剪枝（第 2 轮 N=1）	总成本 −17.9%	★★★★★
执行前缀缓存（h=0.7, η=0.2）	总成本 ≈¥0.9602（−15.5%）【推断】	★★★★☆
早停（少跑轮次）	−10%~20%【推断】	★★★★☆
弱模型/异构评委	总成本 −7.5%	★★★☆☆（不要先砍，多评委是置信度来源）
弱评委 + 剪枝组合	−23.9%	★★★★★

综合缓存、剪枝、早停、级联、批处理等，经济档可把总成本压到单轮的 **3~5 倍**（相对 6.47× 降低约 40%~55%；换算成单模型长跑约 1.2~1.7 倍，均为【推断】，需 A/B 实测校准），具体见 3.7.4 与 3.7.6。

3.7.4 预算分配：多臂老虎机与实用策略

把"每个候选"视为一个臂，拉动 = 安排一次比较/评分，奖励 = 该候选获胜。**成本感知 UCB**：

$$i^*=\arg\max_i (\mu_i+2\sqrt{n}/n_i)/(\text{cost}_i) \text{ 或等价地 } \text{score}_i=\mu_i+2\sqrt{n}/n_i-\lambda\cdot \text{cost}_i$$

Thompson Sampling（预算版）：对每个候选维护 $\text{Beta}(\alpha_i,\beta_i)$，每轮抽样 $\theta_i(\alpha_i,\beta_i)$，选 $\max\theta_i$ 的候选安排比较，观测胜负更新参数，每 8 次重算 P(best)，≥0.95 即停。

实用分配：冷启动每臂至少 2 次（占预算约 20%）；主分配集中到 Top-2 与挑战者；每 10 次强制给最冷门臂 1 次（防早熟）；Top-2 差距 < 噪声时只在两者间加评（配合位置互换）；预算不足一轮时直接输出 Top-1 + 置信度。

3.7.5 停止准则

每轮结束计算 5 个信号：剩余预算 B_rem、后验 P(best)、相邻轮分数变化 Δ_t、排序稳定性 τ_t、SPRT 状态。停止条件（满足任一，但必须区分"确信停"与"预算停"）：

$$\text{STOP}:= (P(\text{best})\geq 0.95) \text{ 或 } (B_{rem}<C_{next}) \text{ 或 } (t\geq T_{max}) \text{ 或 } (|\Delta_t|<\delta \text{ 且 } |\Delta P|<\delta \text{ 连续 2 轮且 } \tau_t\geq 0.9)$$

- **置信停**：输出 Top-1 + 后验概率 + 置信区间；
- **预算停/轮数停**：输出 Top-1 + **未决清单**（Top-2 差距、达到 0.95 还需多少预算估计），并标注"未达置信阈值"；
- **支配停**：某候选 Pareto 支配其余且差距 > 噪声，直接跳过剩余轮；
- **失败停**：硬约束全部失败（无候选通过安全/格式门），报警并回退人工规则，不输出"最优"。

Beta-Bernoulli 后验算例（3 候选，每轮每候选新增 6~8 次成对比较）：

轮次	后验 (A,B,C)	P(A best)	P(B best)	决策
R1	Beta(10,4), Beta(8,6), Beta(5,9)	0.787	0.202	继续
R3	Beta(21,7), Beta(16,12), Beta(8,20)	0.926	0.074	继续
R4	Beta(28,8), Beta(20,16), Beta(9,27)	**0.980**	0.020	**停止，A 胜出**

SPRT 算例（挑战者优势 $p_1=0.75$ vs $p_0=0.5$，$\alpha=0.05$，$\beta=0.20$）：接受 H1 边界 $((1-\beta)/\alpha)=16=2.7726$；n=16、k=13 时 $=3.19>2.77$ → 停止接受"挑战者更优"；Wald 近似期望样本量仅约 16.4（H1 下），少于固定 24 次的均匀方案。

默认参数：$T_{max}=3$、$\delta=0.01$、$\tau\geq 0.9$、P(best)≥0.95（高风险任务可提到 0.99）、α 阈值见 3.4.4。

3.7.6 与"单一大模型长时间集群式跑"的对比框架

用户担心"来回联动的 token 会追上集群模式"。正确的比较对象不是"单轮"，而是"单模型 self-refine 长跑"：

$$C_A=\Sigma_t[N_{tG}+N_t mE+N_{tK}tR]+Fuse+Reg,\quad C_B=T_b mE_B+T_bR_B+重试成本$$

质量模型（需实测标定，不可直接外推）：$Q_A(N,T)=Q^A_\epsilon fty-a_AN^{-\gamma_A}$，$Q_B(T_b)=Q^B_\epsilon fty-a_BT_b^{-\gamma_B}$。

选优判据（三选一，按场景）：

1. 预算约束下最大化质量：$\text{argmax } Q \text{ s.t. } C \leq B$；
2. 质量门槛下最小化成本：$C \text{ s.t. } Q \geq Q_0$；
3. 增量收益 > 机会成本：**当且仅当** $(Q_A-Q_B)V>C_A-C_B$ **时选多方案**（V 为质量点价值，可由返工人力/时间折算）。

多候选更优的条件：任务存在多个合理解法；单模型自迭代 2 轮内接近饱和；候选差异能被评委稳定区分（$\alpha \geq 0.8$）；质量增益价值超过约 2.16× 的成本差。**单模型长跑更优的条件**：唯一正确答案、多样性收益低；预算极紧/延迟敏感；评委一致性 < 0.75（多花钱买噪声）。**公平比较四条硬要求**：等 token 预算、等任务集、同一评测协议、报告置信区间；并做两个消融（多候选 vs 同候选多采样；多评委 vs 单评委多次）。

本节小结：成本模型给出的结论不是"多方案便宜"，而是"**多方案的钱主要花在执行上，所以剪枝、缓存、早停的收益最大；评委不能先砍，评审是置信度的来源**"；对用户而言，产品必须按任务价值分档，而不是所有任务都开满。

3.8 双形态架构：Skill 与 Orchestrator 的分工（解决"头脑与身体脱节"）

3.8.1 脱节从哪里来

用户原始想法 3 的担忧是对的：如果"外部 LLM 只负责写提示词、执行模型只负责干活"，会出现四个断点：

断点	表现	后果
目标不一致	外部 LLM 不知道执行模型的上下文窗口/工具限制	提示词"看起来专业"，执行不了
上下文不同步	执行模型的中间状态不回传或回传成自然语言	评审只能凭想象打分
反馈不结构化	执行结果散落在聊天记录里	无法比较、无法复现
责任边界不清	两边都能"改需求"，没有唯一事实源	来回甩锅、任务拖沓

3.8.2 双形态分工与数据契约

	Skill（内嵌）	Orchestrator（外置）
定位	执行端"最小自我修正"	跨模型"决策与仲裁"
职责	改写提示词 → 自检 4 项 → 最多 2 轮返工	需求编译、多候选、多模型执行、评审、决策、融合
状态	SRD-lite（目标 + 验收摘要）	SRD 全量 + Evidence + ScoreCard + Prompt DAG

	Skill（内嵌）	Orchestrator（外置）
协议	工具调用 / Skill 包（可复用 Skill Hub）	MCP（工具接入）+ A2A（Agent 委派）+ REST
输出	改进后的提示词 + 自检结论	交付包（方案 + 证据 + 评审报告 + 成本账单）
成本	约 1.2~2× 单次调用（估算）	算例约 2.16× 单模型长跑；经济档约 1.2~1.7×【推断】

关键设计：编排器**只做三件事**——定义目标与验收（SRD）、分配预算、仲裁分歧；执行端 Skill **拥有**"在单会话内怎么改"的自主权。两者之间只传结构化对象，不传"你觉得怎么样"这种自然语言寒暄。这样"头脑"不越权指挥细节，"身体"不会脱离目标乱跑。

3.8.3 模型调用的合规与数据边界

模型获取遵循"官方渠道、可审计、数据可控"三条原则：

维度	规则
调用渠道	仅通过官方 API / 云服务（Pangu、Qwen、DeepSeek 等）或本地部署模型调用；不接入非官方页面、逆向接口或未授权渠道
数据流向	记录每次调用的数据流向与保留策略；敏感代码/数据默认只走本地或专属实例，传输前脱敏
成本记录	记录模型、版本、单价、token 与费用，写入 Token Ledger；价格变更时同步更新台账
账号与额度	使用组织/个人正式账号与官方计费方式，遵循服务条款；不使用来源不明的额度或代理
降本手段	模型路由、前缀缓存、批处理、本地小模型初筛、候选剪枝与早停；成本优化不牺牲合规与数据安全

3.8.4 形态选择决策表

任务特征	推荐形态	理由
单轮小改动、延迟敏感	Skill	秒级、低成本
多文件/多方案/需评审	Orchestrator	需要证据与统计决策
涉密/私有代码	本地 Orchestrator + 本地模型	数据不出域
演示/答辩	Orchestrator + 缓存回放	稳定、可解释
批量任务	Orchestrator 经济档 + 批处理	成本优先

本节小结：双形态不是"两个产品"，而是"同一套协议的两个执行位置"：Skill 负责让执行端不脱节，Orchestrator 负责让决策不短视；模型调用只在合规边界内进行，成本通过工程手段控制。

3.9 openJiuwen 生态使用说明与部署

> 官方明确：推荐但不强制使用 openJiuwen WorkSwarm / AgentOS；**使用 openJiuwen 生态（WorkSwarm、开源组件、社区工具链）的作品，在评审中将获得"技术方案"维度加分**。本节说明"用了什么、怎么用、哪些自研"，避免"只字未提生态"或"套壳"两种极端。

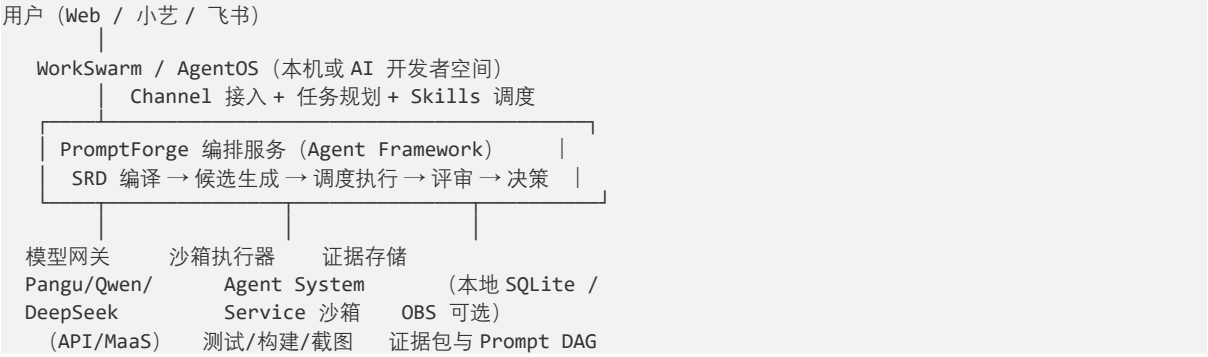
3.9.1 组件映射：复用底座，聚焦方法层

PromptForge 模块	复用的 openJiuwen 能力	自研部分
Agent 编排与任务规划	**WorkSwarm** 任务规划、Swarm 协同；**Agent Framework** 多智能体协同与执行引擎	Prompt DAG 状态机、预算调度策略
需求编译 Agent	**Agent Studio** 工作流编排、在线调试	SRD Schema、澄清问题生成与完整性评分
候选生成 Agent	**Agent Studio** Prompt 调优、在线调试	假设矩阵采样、PIR 组装、多样性去重
执行 Agent	**Agent System Service** 安全隔离沙箱、CLI 工具、标准化文件系统；**Agent Runtime** 分布式执行	模型网关适配层、证据包 Schema、执行动作标准化
评审 Agent	Agent Framework 多 Agent 协同；DeepAgents / JiuwenSwarm 多实例协同	盲评/换序协议、偏差校正、BT/Elo 聚合、一致性检验
决策/融合 Agent	JiuwenSwarm 协同、Agent 自演进（识别轨迹异常、Skills 演进）	TOPSIS/Pareto/HV、集合覆盖/背包、McNemar 回归门
成本管家	Agent Framework 算力亲和调度、记忆子系统；WorkSwarm 上下文瘦身	Token Ledger、UCB/Thompson 预算分配、早停判据
轻量 Skill 形态	**WorkSwarm Skills** 自主演进 + **Swarm Skills Hub** 分发	Skill 的提示词模板与自检规则
入口与触达	WorkSwarm **Channel**（小艺/飞书接入）、浏览器操控	Web 对比看板与会议模式

自研边界（诚实声明）：PromptForge 不重复实现 Agent 运行时、沙箱、记忆、调度、渠道接入；自研集中在**需求编译（SRD）、证据协议（Evidence Packet）、评审聚合与偏差校正、多准则决策（MCDM）、成本调度、对比可视化**六块——这正是"方法层"的差异化所在。

> 关于"Prompt 自优化"：openJiuwen Agent Studio 已提供 Prompt 调优能力，官方也将"Prompt 自优化"列为生态能力之一。PromptForge 的定位不是"重新发明提示词优化"，而是**在它之上补上"多候选同题竞演 + 执行取证 + 统计决策 + 成本控制"的决策层**；Agent Studio 的 Prompt 调优会作为候选生成策略之一被复用。这个边界写清楚，既拿生态分，也避免"重复造轮子"的质疑。

3.9.2 部署架构



- **开发/演示环境**：`pip install workswarm` → `workswarm-init` → `workswarm-start`（默认 `http://localhost:5173`），PromptForge 作为 WorkSwarm 的 Skill/插件接入；模型走 Pangu/Qwen/DeepSeek API 或本地开源模型。

- **比赛算力**：华为云 **AI 开发者空间**（官方指定入口）承载演示与批量评测；生产化时可选用华为云 OBS 存证据链、APIG 做入口、GaussDB 存版本 DAG（均为可选，不构成依赖）。
- **降级路径**：演示现场网络不通时，切换本地模型 + 缓存回放，保证 Demo 不翻车。

3.9.3 生态叙事（答辩用）

- **用了什么**：WorkSwarm（规划/Channel/Skills）、Agent Framework（多 Agent 协同）、Agent Studio（Prompt 调优/工作流）、Agent System Service（沙箱）、DeepAgents/JiuwenSwarm（协同）、Swarm Skills Hub（Skill 分发）。
- **补了什么**：SRD 需求编译、证据驱动竞演、统计评审与 MCDM 决策、Token 预算调度——这是 openJiuwen 生态当前不覆盖的"决策层"。
- **可复现**：全部组件开源（Apache-2.0 计划），依赖清单、部署脚本、评测数据与实验日志随仓库提交。

本节小结：与 openJiuwen 生态的关系应是"站在巨人肩膀上补一层"：运行时、沙箱、渠道、Skill 分发全部复用，把有限的比赛时间投入到"可验证决策"这个真正的差异化上；这既是技术选择，也是评审叙事。

3.10 安全、隐私与合规设计

风险	设计对策
提示注入（用户输入/仓库内容里藏指令）	把外部内容当**数据**而非指令；系统提示与用户内容强隔离；执行/评委最小权限；输出 schema 校验；危险工具调用需二次确认
不可信代码执行	沙箱隔离、默认禁网、只读基础镜像、资源配额、超时、无宿主机路径访问；执行日志脱敏
密钥与凭证	环境变量/密钥管理服务，不落库；`.env.example` 占位；演示材料与截图中不出现真实密钥
数据隐私	敏感任务强制本地/专属实例（隐私路由）；可配置脱敏；训练/评测数据来源与许可证声明；不使用未授权数据
内容安全	接入华为云内容安全服务做生成内容审核；交付前扫描
开源合规	获奖作品 Apache-2.0、仓库根目录 LICENSE、README/技术方案/依赖声明/部署指南；**第三方代码 ≤ 有效代码量 30%**；Git 提交历史连续可追溯；第三方依赖逐项列名称/版本/来源/许可证/作用
比赛合规	不使用大赛评测集/测试集训练、调参、设计提示词；演示数据脱敏模拟；不伪造测试数据/演示效果
滥用风险	限流、审计日志、用户可删除数据；对攻击系统、越权访问、恶意代码类请求拒答

红队测试计划：把"提示注入、越权工具调用、敏感信息泄露、恶意代码执行、评测数据污染"写成 5 组红队用例，纳入 golden set 与回归门（任一回归直接拒绝发布）。

本节小结：安全不是附加项，而是"项目运行"评分档位里"卓越且安全的架构"的直接得分点；同时比赛把数据/安全合规设为评审前置条件，任何一项失守都可能一票否决。

3.11 机机联动层（Machine Link Layer）

> 本节回应最初想法 2：不只让模型"生成结果"，而是让**规划机、执行机、评审机**通过结构化协议连成闭环。它是本作品从"提示词优化"走向"执行体协同"的关键层，也是后续扩展到浏览器、CI、设备/机器人的接口。**L0 是实测，L1/L2 是设计目标，不是当前承诺。**

3.11.1 为什么需要机机联动

单模型的一次生成只能看到"文本结果"，看不到代码是否运行、接口是否返回正确、页面是否渲染、设备动作是否完成。要让"选方案"真正成立，必须让执行体把观察结果回传，让评审体基于同一份观察做判断，再让规划体决定"继续、修复、融合或停止"。这不是让多个模型闲聊，而是让**每一次执行都成为下一次决策的证据**。

3.11.2 三层机器与一个总线

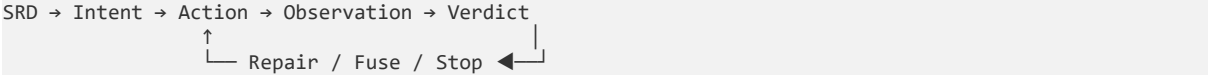
角色	职责	典型实现
规划机 Planner	把 SRD 拆成 Intent，分配预算/权限，决定下一步	Orchestrator / WorkSwarm WorkflowAgent
执行机 Executor	接收 Action，在真实环境执行并回传 Observation	编码 Agent、CI Runner、浏览器自动化、设备适配器
评审机 Judge	规则校验 + 异构 LLM 盲评，产出 Verdict	测试/编译/lint + 多厂商评委
联动总线 Link Bus	路由、鉴权、重试、审计、预算与追踪	MCP / A2A / HTTP + Agent System Service 通信

四类结构化消息（**IAOV**）：

- **Intent**：要达成什么（引用 SRD 与验收标准）；
- **Action**：允许做什么（工具、参数、权限、超时、预算、幂等键）；
- **Observation**：实际发生了什么（Evidence Packet、日志、截图/关键帧、哈希、环境指纹）；
- **Verdict**：如何判定（规则结果、评委分、置信度、下一步建议）。

每条消息都必须带 `task_id` / `candidate_id` / `endpoint_id` / `contract_hash` / `trace_id` / `permission_scope` / `budget`；禁止只传自然语言聊天记录。自然语言可以有，但只能作为 Observation 中的非结构化附件，不能作为决策依据。

3.11.3 联动闭环



1. **首轮**：规划机把 SRD 编译为 N 个候选的 Action，附带验收标准与预算；
2. **执行**：执行机在沙箱/CI/浏览器/设备上执行，回传 Observation（Evidence Packet）；
3. **评审**：规则引擎 + 异构评委基于同一份 Evidence 盲评，产出 Verdict；
4. **决策**：决策器按硬约束、预算、置信度选择"修复 / 融合 / 停止"；
5. **回环**：修复或融合生成新的 Action；父代、证据与结论写入 Prompt DAG，可回放。

3.11.4 跨执行体的四个层级

层级	范围	状态
L0 单机回读	本机子进程 + pytest + Evidence Packet + 决策卡	**已实现**：demo-004 真实跑通 A/B 对照
L1 跨进程/跨模型联动	规划机、执行机、评审机分属不同进程/模型，MCP/A2A 传输	设计目标：接口已具备，未做稳定性验证
L2 跨环境联动	CI Runner、浏览器、IDE 插件等执行端点，能力清单 + 幂等 + 回放	设计延展：先 Coding 场景，不承诺本期完成
L3 跨设备/物理执行	机器人/设备适配器，安全包络（速度/力/空间/急停）+ 人工确认	长期研究方向，不作为 Coding 赛道交付承诺

> 为什么先 L0/L1：Coding 的证据最硬，执行体也最容易标准化（进程/容器/CI）。L2/L3 不是"以后再说"，而是把同一套 IAOV 协议和状态对象延伸到更多执行体；没有 L0 的证据闭环，直接做设备联动只会得到更贵的黑箱。

3.11.5 为什么难，怎么解

难点	风险	设计应对
异构接口	每个执行体能力/参数不同	Capability Manifest + 统一 Action schema + 适配器
状态同步	多轮后上下文漂移、目标丢失	单一事实源（SRD/Evidence/DAG）+ trace_id + 每轮重放
权限与安全	执行体越权或危险动作	最小权限、沙箱、审批门、安全包络、可撤销令牌
成本/延迟	来回联动导致 token/时延上涨	Token Ledger + 预算/轮数上限 + 早停 + 缓存
可复现	执行环境漂移、结果不可比	环境指纹、依赖锁、证据哈希、离线回放
非确定性	模型/网络/设备波动	多次采样 + 置信区间 + 人工复核

这六个难点也是设计文档把它单列一章、而不是塞进"工具调用"小节的原因：**联动不是多一次调用，而是多一层状态与责任边界**。

3.11.6 与现有机制的关系

- **SRD → Intent**：需求规格是联动的起点，验收标准决定 Action 的边界；
- **Execution Contract → Action**：把"允许做什么"写进契约，而不是让执行体自由发挥；
- **Evidence Packet → Observation**：机器可读结果是联动的硬证据，不是截图给人看；
- **ScoreCard → Verdict**：评审结论必须引用证据，否则不进入决策；
- **Prompt DAG → 联动轨迹**：每次 Action、修复、融合的父代与证据可回放。

因此，机机联动不是另起一套系统，而是把 3.1–3.6 的状态对象用 IAOV 协议串起来；v0.1 的"执行 → 证据 → 决策"是它的第一段，后续扩展执行端点即可。

3.11.7 边界

- 不做无协议、无预算、无权限的**开放式多机聊天**；

- 不做无人工确认的高风险**物理动作**；
- 不以"联动轮数"作为效果指标，而以**证据完整率、回放一致率、单位采纳方案成本、人工干预率**衡量；
- 任何一层联动都必须能"停下来、查得到、复现得了"，否则不进入下一层。

4. 预期体验（So What）

前面三章讲的是系统"是什么、怎么转"；这一章把机制翻译回人的感受，并给出**可观测的验证指标**，避免"我觉得很好用"式的自嗨。

4.1 三类用户与前后对比

用户	现在怎么做	痛点	用 PromptForge 之后
独立开发者 / 学生	想到什么问什么，来回改十几轮	不知道提示词差在哪，方案返工多，token 与时间成本不可见	输入关键词 → 拿到 3 个带证据的方案与推荐理由，10 分钟内确定方向
企业研发团队	资深工程师手写方案，评审靠经验	方案质量依赖个人水平；评审意见难量化；模型供应商锁定	多方案对比看板 + 评审报告 + 成本账单，评审会从"拍脑袋"变成"看数据"
非专业"公民开发者"	不会写提示词，只能套模板	需求说不清，AI 输出不可控	需求编译器主动澄清、自动补全验收标准，把"不会问"变成"会问"

4.2 分阶段体验

新手期（0~1 小时）：零配置。输入一句话，跟着 3 个澄清问题走，默认模板与默认预算即可产出可用结果。体验目标是"第一次就拿到 3 个像样的方案，而不是 1 个半成品"。

熟练期（1~2 周）：会调三样东西——候选数量 N、评委组合、预算档位；会用"会议模式"投屏对比、给方案打批注、锁定某些假设；会看成本-质量曲线决定"要不要再迭代一轮"。

精通期（1 个月以上）：接入团队内部规范与历史项目知识库，自定义评审 Rubric 与权重，接入私有模型/本地推理，把 PromptForge 当成团队的"方案决策基础设施"；对高价值任务使用多轮融合，对低价值任务使用 Skill 单机内循环。

4.3 三个典型场景

> 以下为叙事示例（用于说明体验流程），其中的方案数量、耗时与成本均为示意，不是实测数据。

场景 A：独立开发者（关键词 → 可运行原型）

输入："做一个校园社团活动报名的网页，能导出名单。" 系统澄清 2 个问题（部署环境？是否需要审核流程？）→ 生成 3 个方案（单体 Next.js / 前后端分离 / Serverless）→ 并行执行并跑测试 → 对比看板显示候选 B 质量更高但成本也更高，候选 A 成本最低且测试全过 → 推荐 A，附 B 的两个优点（审核流程状态机、导出模板）供融合 → 用户点"融合" → 回归测试通过 → 导出仓库 + 提示词 + 报告。

场景 B：团队方案评审会（把"吵架"变成"看数据"）

投屏"会议模式"：左侧是候选并排 diff，右侧是雷达图与成本-质量散点；每个争议点标注"证据来源"（测试用例、lint 报告、性能数据）；评委打分不一致的维度高亮显示。团队当场选定方案或指定融合项，系统生成会议决议与版本记录。

场景 C：IDE 内 Skill（随手的质量提升）

在 DSH/Codex 里写代码时，输入"把'用户登录'这个需求改写成更好的提示词"，Skill 在单会话内完成：重写提示词 → 自检 4 项 → 若不合格自动再改 1 轮 → 返回改进后的提示词与"为什么这样改"的一句话说明。交互为秒级返回，成本约为单次调用的 1.2~2 倍（估算，待实测）。

4.4 10 分钟现场 Demo 脚本（答辩用）

时间	动作	展示要点	兜底方案
0:00–1:00	输入关键词"做一个带登录的待办事项 Web 应用"	需求编译器提出 2 个高价值澄清问题	预置录屏
1:00–2:30	展示 4 个候选的"假设差异卡"	不是同义改写，而是架构假设不同	用缓存结果
2:30–5:00	并行执行 + 证据包实时滚动	测试通过率、lint、性能、截图	展示已完成的证据包
5:00–7:00	多评委盲评 + 一致性系数 + TOPSIS 排序	评分有置信区间，评委分歧可见	固定演示数据集
7:00–8:30	对比看板 + 会议模式批注 + 融合	优点抽取、冲突检测、回归测试	单候选融合
8:30–10:00	交付包与成本账单	"同一任务比手工试错省 X 轮、成本 Y 元"	展示历史账单

4.5 可观测指标与验证计划

指标	定义	目标（设计目标，需 A/B 实测校准）	采集方式
一次通过率	首次交付即被采纳、不需返工的任务占比	较基线提升（幅度以实测为准）	交付记录 + 用户标记
平均返工轮数	用户要求"再改一版"的次数	目标 ≤2 轮（以实测为准）	会话日志
方案采纳率	推荐的 Top-1 被用户选中的比例	≥ 60%	用户选择记录
评审一致性	多评委 Krippendorff's α	≥ 0.6	评分矩阵计算
单位任务成本	每个任务的 token 费用	≤ 同等质量单模型试错的 1.5 倍	Token Ledger
需求澄清完成率	用户完成澄清问答的比例	≥ 80%	交互埋点
用户主观质量分	5 分制满意度	≥ 4.0	交付后问卷

> 验证方法：准备 30 个覆盖代码、文档、多模态的任务集，做 A/B——A 组"直接问一个大模型并人工来回改"，B 组"PromptForge 一次交付"；记录返工轮数、总 token、耗时、人工评分。**以上全部为设计目标，不是当前实测结果；实测数据将记录在附录 G 与 DEMO 报告中。**

4.6 设计风险与验证指标

风险	可能表现	观测指标	缓解措施
评审噪声导致选错方案	推荐方案与人工评审结论不一致	Top-1 与人工评审一致率	异构评委 + 位置互换 + 可执行验证占比提高
多候选成本失控	单任务成本超预算 2 倍以上	成本/质量曲线、预算命中率	预算控制器 + 早停 + 级联评审
融合导致回归	融合后测试通过率下降	融合前后测试通过率差	融合强制回归 + 失败回退最优单方案
用户不愿澄清	直接跳过问题，SRD 假设错误	澄清完成率、假设致返工率	每题给默认选项，最多 3 问
模型 API 不稳定	某厂商限流/宕机导致任务卡住	任务成功率、降级率	多厂商路由 + 备用模型 + 断点续跑
评委"自我偏好"	生成某候选的模型给它打高分	评分与模型来源的相关性	匿名化 + 评委不评自己生成的候选
演示环境不可控	现场网络/沙箱故障	演示成功率	本地缓存回放 + 录屏兜底

本节小结：PromptForge 的体验目标不是"一键出神作"，而是**把不确定性前移到"选方案"阶段、把确定性留给"执行验收"**；用户的感受应该是"这次一开始方向就对了"，而不是"AI 又给了我一个看起来不错的半成品"。

5. 可行性评估与优化建议

本章回答三个问题：**这个作品做不做得出来？值不值得做？用户原始想法里哪些该保留、哪些该改造、哪些该放弃？**

5.1 总览结论

编号	用户原始想法	核心价值判断	技术可行性	工程可行性	成本	合规风险	结论
想法 1	做"关键词→高质量提示词→输送给 Codex/DSH/GPT"的中转界面；多 LLM 生成多方案；用数学模型对比方案、可视化、开会讨论、选最优、融合优点	高	高	高	中	低	**保留为主线**，即本方案的 Orchestrator 核心
想法 2	让 LLM 与执行大模型"机机联动"；把代码/图像/视频的执行结果转成机器可读证据，供 LLM 评审与回环优化	高（"执行结果回读 + 评审闭环"是核心价值）	高	高	中	低	**保留并工程化**：结构化证据回读（Evidence Packet）+ 可选多模态证据（关键帧/OCR/ASR 工具提取）+ 多评委评审
想法 3	纠结"外部 LLM 编排"还是"做成大模型 Skill 自己优化"；希望降低使用成本，让人人都用得起	高（架构取舍问题真实存在）	高	高	低~中	低（仅用官方 API/本地模型）	**双形态**：Skill 负责单会话内循环，Orchestrator 负责跨模型决策；成本靠路由/缓存/本地模型控制
想法 4	机机联动来回导致 token 暴涨，可能接近"集群模式"消耗，需要优化 token	高（成本是产品能否普及的关键）	高	高	—	低	**保留并升级为核心竞争力**：Token Ledger + 预算调度 + 早停/剪枝/级联/缓存

5.2 技术可行性（按模块）

模块	依赖的成熟技术	成熟度（TRL，评估）	主要风险	备选方案
需求编译 SRD	结构化输出、槽位抽取、澄清问答	8	需求本身不可判定	模板 + 人工编辑
多候选生成	Prompt 组合、多样性去重、Self-Refine	8	候选同质化	反事实候选、假设矩阵强制差异
模型网关	OpenAI-compatible API、LiteLLM/vLLM 类方案	9	厂商 API 变更	多厂商适配 + 自建推理

模块	依赖的成熟技术	成熟度（TRL，评估）	主要风险	备选方案
执行沙箱	Docker/微虚机、测试框架、Playwright	9	不可信代码安全	禁网 + 最小权限 + 快照回滚
证据采集	测试/lint/SAST/性能/截图/ASR/OCR	9	证据维度覆盖不足	证据模板可扩展
多评委聚合	LLM-as-a-Judge、Bradley-Terry、一致性检验	7	偏差与噪声	异构评委 + 盲评 + 规则评审兜底
多准则决策	TOPSIS、Pareto、bootstrap	9	权重主观	权重可配置 + 敏感性分析
融合与回归	优点抽取、diff 合并、回归测试	6	融合冲突/回归	只在低耦合优点上融合，失败回退
预算与停止	Token 计量、UCB/Thompson、SPRT	7	收益函数估计不准	先启发式后自适应，人工兜底
可视化	雷达图/散点/热力图/并排 diff	9	信息过载	分层视图（摘要/详情）
Skill 形态	Agent Skills / MCP 工具调用	7	各 Agent 规范不统一	适配层 + 提示词模板降级

结论【推断】：除了"融合与回归""预算与停止"需要一定实验调参外，其余模块都有成熟工程实践；整体技术风险可控，属于"集成创新"而非"从零发明"，适合比赛周期。

外部证据（说明"不是拍脑袋"）：

- 提示词自动优化有效：APE 在 24 个任务中 19 个达到或超过人工指令（arXiv:2211.01910）；OPRO 比人工 prompt 在 GSM8K 上 +8%、BBH 上 +50%（arXiv:2309.03409）；DSPy 编译后 +25%/+65%（arXiv:2310.03714）。前提是"有可自动打分的验证集"，所以 SRD 的验收标准必须可执行。
- 多评委可用但需去偏：GPT-4 评委与人类偏好一致率 >80%（arXiv:2306.05685），但位置偏差（arXiv:2305.17926）与自我偏好（arXiv:2404.13076）真实存在。
- 可执行验证是硬信号：SWE-bench 用真实测试判定补丁（arXiv:2307.03172），CodeT 用"生成测试→执行→投票"提升选码（arXiv:2207.10397）。
- 风险有据可查：代理指标反复优化会先升后降（arXiv:2309.17453）；多智能体辩论成本约 6~9 倍且等算力下收益缩水（arXiv:2311.17371）——这两条直接决定了本方案"早停 + 竞演而非辩论"的设计。
- 成本优化手段成熟：级联（FrugalGPT）、路由（RouteLLM）、压缩（LLMLingua）、缓存（OpenAI Prompt Caching）。

5.3 工程可行性（比赛周期视角）

MVP（必须做出来）：需求编译 + 3 候选生成 + 2 个模型执行（一个商业 API + 一个开源/本地）+ 证据包（测试/lint/构建）+ 3 个 LLM 评委 + TOPSIS 排序 + 对比看板 + 交付包导出 + 成本账单。

增强项（有时间再做）：融合与回归、会议模式批注、Pareto 前沿、预算调度器、Skill 形态、openJiuwen WorkSwarm 部署、本地模型推理、历史知识库。

明确不做：模型训练、通用评测榜单、无限制自主 Agent、生产发布流水线接管。

阶段	主要交付	预估工作量（人·周，示例假设）	风险缓冲
M0 需求与规范	SRD schema、证据包 schema、评分 Rubric、Demo 任务集	1~2	低
M1 核心闭环	编译器 + 候选生成 + 网关 + 沙箱 + 证据包	3~4	中（沙箱环境）
M2 评审决策	多评委 + 一致性 + TOPSIS + 看板	2~3	中（评审噪声）
M3 打磨与材料	融合、成本账单、Demo、创意 PPT、技术报告、答辩	2~3	低
合计	—	**8~12 人·周**	—

> **按真实赛程倒排**：以 2026-10-20 为起点，**10-31 前**必须完成报名/实名/组队并冻结创意方案；**11-10 前**提交创意 PPT（此时至少要有 M1 的可运行原型与 1 次端到端演示录屏）；**11-25—12-27** 是 DEMO 开发窗口，完成 M2/M3、3 分钟视频、技术报告与开源仓库准备。详细日程见 §7.2 与附录 E。

5.4 成本可行性

多候选 + 多评委必然比"单模型单轮"贵，关键在于**贵多少、值不值、能不能压**。以下数字来自 3.7 的【算例】（参数为假设值，非实测）：

- 朴素实现（3 候选 ×2 轮 ×4 评委，全部用大模型）在算例参数下约为单模型单轮的 **6.47 倍**（参数为假设值，非实测）；
- 经济档（候选预评审剪枝 + 小模型初评 + 缓存 + 早停 + 按证据难度分配预算）在算例中可压到单轮的 **3~5 倍**，约单模型长跑的 **1.2~1.7 倍**（【推断】，需 A/B 实测；详细模型与算例见 3.7）；
- 若质量收益（返工减少、方案采纳率提升）带来的时间/人力节省超过成本增量，则经济上成立。用 3.5 的决策不等式判断：当 $\Delta Q/Q_{base} > k \cdot \Delta C/C_{base}$ 时才值得启用多候选（k 为用户可配置的"质量-成本汇率"）。

> **诚实结论**：对"写个脚本"这类低价值任务，多候选不划算，应默认走 Skill 单机模式；对"方案选型/架构设计/长周期项目"这类高价值任务，多花约 2~5 倍 token 换一次正确的方向，ROI 显著为正。产品必须**按任务价值自动切换档位**，而不是所有任务都开满。

5.5 合规、伦理与可持续性

风险点	具体问题	结论与对策
模型调用合规	非官方渠道/未授权代理存在服务条款与数据风险	仅使用官方 API、云服务或本地部署模型；记录数据流向与计费；不使用来源不明的额度或代理

风险点	具体问题	结论与对策
提示注入	用户输入/仓库内容可能含恶意指令，诱导评委或执行模型	把用户内容视为不可信数据；评委与执行器分权；输入过滤 + 输出校验 + 最小权限
数据隐私	代码/文档上传第三方模型	隐私路由：敏感任务强制本地/专属实例；可配置脱敏
内容安全	生成内容可能违规	接入华为云内容安全服务；交付前扫描
开源许可	执行模型生成的代码可能引入 GPL 等许可冲突；获奖作品要求 Apache-2.0 开源、第三方代码 ≤30%	依赖扫描 + 许可证检查写入证据包；从第一天起保持 Git 历史连续、依赖声明完整；不使用来源不明的代码
比赛合规	不得使用大赛评测集/测试集训练、调参、设计提示词；不得伪造数据/演示效果；不得同一作品多队/多赛道重复提交	评测集自建并公开来源；Demo 数据脱敏；提交材料与仓库一致
学术诚信/比赛规则	使用第三方模型与开源组件需声明	交付包与答辩材料中列明依赖与许可

5.6 对用户四个原始想法的详细评审

想法 1：中转 + 多提示词 + 数学对比 + 可视化 + 融合

保留，作为作品主线。 这正是"提示词编译器 + 多方案决策"的核心。需要补三块用户原想法中没有的工程约束：

1. **"中转站"要升级为"带证据的网关"**：只转发没有壁垒，转发 + 执行 + 取证 + 计费 + 审计才有。
2. **"数学模型"要落到可解释的指标与统计**：单纯算一个综合分容易被质疑；需要 TOPSIS/Pareto + 置信区间 + 敏感性分析（见 3.5）。
3. **"融合优点"要工程化**：优点必须先结构化为"可合并单元"，融合后必须回归测试，否则"融合"会变成"缝合怪"。建议一期只融合低耦合优点（测试用例、边界处理、文档、提示词片段），架构级融合放二期。

想法 2：机机联动 + 执行结果回读评审

不放弃，升级为独立设计层（3.11），并作为第五个创新点。 最初想法里的"机机联动"如果只理解成"让模型互相聊天"，容易变成昂贵的开放式辩论；真正有价值、也能工程化的是：**规划机 → 执行机 → 评审机** 通过 IAOV 结构化消息形成闭环，执行结果以 Evidence Packet 回读，评审结论再驱动修复、融合或停止。落地分三层：

- **L0 单机回读（已实测）**：同一台机器上完成"生成 → 沙箱执行 → pytest → Evidence → 决策"，demo-004 已跑通 A/B 对照；
- **L1 跨进程/跨模型联动（设计目标）**：规划机、执行机、评审机分属不同进程/模型，用 MCP/A2A 交换 IAOV 消息；
- **L2 跨环境/设备联动（设计延展）**：CI、浏览器、IDE、设备/机器人作为执行端点，加能力清单、幂等动作、安全包络与回放。

它比开放式多智能体辩论更适合 Coding：代码的真相在测试、编译器和运行环境里，不在聊天里；同时保留"多机协同"的想象空间（浏览器、CI、机器人执行体）。边界与难点见 3.11，演进路线见 7.6。

想法 3：外部 LLM 编排 vs 大模型 Skill

两者都要，但分工明确。 用户担心的"头脑（LLM）与身体（大模型）脱节"是真实问题，根因是：目标不一致、上下文不同步、反馈不结构化、责任边界不清。解法是**双形态 + 单一事实源 + 协议化**（见 3.8）：

- **Skill** 解决"单会话内、低延迟、低成本"的自迭代，让执行端不再"无脑"；
- **Orchestrator** 解决"跨模型、多方案、可审计、带预算"的决策；
- 两者用 SRD/Evidence 共享状态，执行端回传证据，编排器只做仲裁。

模型获取与成本：所有模型调用走官方 API/云服务或本地部署；"人人用得起"通过模型路由、缓存、批处理、本地小模型与按任务价值分档实现。

想法 4：Token 成本优化

保留并升级为核心卖点。 用户担心"来回联动导致成本追平集群模式"，这个担心合理，但结论不是"少联动"，而是"**会算账地联动**"：Token Ledger 计量 + 预算分配 + 早停 + 剪枝 + 级联 + 缓存 + 蒸馏。要敢于在答辩中给出"成本-质量"曲线，而不是只讲功能。

5.7 八条可执行优化建议

1. **先做窄场景：**一期只做"Web/脚本类代码任务"，把证据链做深，不追求全品类。
2. **把"证据"做成产品壁垒：**同样的多方案对比，别人给评分，你给"测试通过率 + lint + 性能 + 可复现哈希"。
3. **评委异构化：**至少覆盖 3 家不同厂商模型 + 1 个规则评审，避免"同源模型互相吹捧"。
4. **预算档位化：**经济/标准/深度三档，默认经济档，按任务价值自动推荐。
5. **融合保守化：**只融合可验证的低耦合优点，架构级融合必须有回归测试与人工确认。
6. **把 Skill 作为获客入口：**轻量、免费/低成本，让用户先感受到"提示词变好了"，再引导到 Orchestrator。
7. **数据闭环：**记录用户最终采纳与修改，反哺权重与 Rubric（注意隐私与授权）。
8. **讲一个可验证的故事：**比赛材料里放"同一任务：传统方式 N 轮 vs PromptForge 2 轮"的对照实验，比讲架构更有说服力。

5.8 竞赛可行性：按官方维度自评

官方评审分两个阶段、两套维度，且官方创意 PPT 模板附录给了 1–5 分量规。按"证据 → 自评 → 补强动作"对一遍（自评分数为目标值，非官方评分）：

阶段/维度	我们的证据	目标分	补强动作
创意·创新性	证据驱动多方案决策、异构评委去偏、成本感知编排、双形态；竞品在"可审计决策层"有空位	4/5	用 1 页对比表讲清"和单模型 self-refine 的区别"
创意·实用性	减少返工轮数、降低提示词门槛、团队评审可量化	4/5	补 3 个真实任务的前后对照
创意·技术可行性	成熟模块 + 论文依据 + 开源底座（WorkSwarm/AgentOS）+ 8~12 人周计划	4/5	11-10 前给出可运行原型录屏
创意·场景价值	Coding 赛道最痛、可扩展到办公/科研	4/5	量化"一次返工 = 多少时间/token"
DEMO·完成度	端到端闭环 + 沙箱 + 降级方案	4/5	演示环境双备份
DEMO·用户体验	对比看板、会议模式、成本账单、决策卡	4/5	请 3 位外部开发者试用并记录
DEMO·技术难度	多模型网关、证据协议、BT/TOPSIS、预算调度	5/5	技术报告给出公式与复现步骤
DEMO·商业价值	三层商业模式 + ROI 折算	3~4/5	补充目标客户访谈/报价假设
模板量规·项目运行	可运行原型 + 安全架构（沙箱/权限/审计）	4~5/5	演示"卓越且安全的架构"证据链
模板量规·创新与创造力	五个创新点 + 与 MAD/单点工具的边界	4/5	突出"机制创新"而非"模型能力"
模板量规·对现实世界的影响	"让每个开发者请得起 AI 专家评审团"	4/5	用一个用户故事讲影响
模板量规·主题一致性	多智能体协同（智体共生）+ 现实场景创新应用	4~5/5	封面第一页就点题

最大的竞赛风险：创意阶段只有 PPT、没有可运行 DEMO 时，容易被评为"想法大、实现少"。对策：**11-10 前必须完成"可运行的最小闭环"并录屏**（哪怕只支持 1 个模型 + 2 个候选 + 规则评审），把"项目运行"这一档先拿到 3 分以上，再在 DEMO 阶段冲 4~5 分。详见 §7.2。

本节小结：四个想法中，1 和 4 直接保留为产品核心，3 用"双形态"化解，2 的"机机联动 + 执行结果回读"保留并工程化；整体技术可行、工程可控、合规风险可通过取舍规避。按官方赛程与评审维度倒排，本作品在 10-31 报名截止、11-10 创意提交、12-27 DEMO 截止的节奏下具备可交付性，最大风险是"创意阶段缺可运行证据"，已用"11-10 前最小闭环 + 录屏"的对策覆盖。

6. 创新点与差异化

6.1 五个创新点

创新点 1：证据驱动的多方案决策（Evidence-based Portfolio Decision）

现有提示词工具优化的是"文本"，PromptForge 优化的是"决策"：每个候选必须真正执行并产出可验证证据（测试、lint、性能、安全扫描、截图），评委只能在证据上打分，最终用 TOPSIS/Pareto 做多准则决策，并给出置信区间。把"哪个提示词更好"从主观判断变成可复现的测量。这套思路有直接的技术依据：SWE-bench 用仓库真实测试判定补丁是否有效（arXiv:2307.03172），CodeT 用"生成测试→双执行→投票"低成本提升选码质量（arXiv:2207.10397）；而"重复采样能提升覆盖率、但没有可靠验证器就无法兑现收益"的结论（arXiv:2407.21787）恰恰说明：**验证器/选择器才是瓶颈，也是本方案的壁垒**。

创新点 2：异构多评委 + 偏差校正 + 统计收敛

不是"让 LLM 打个分"，而是把 LLM-as-a-Judge 做成可审计的评审系统：评委来自不同厂商/规模、匿名盲评、位置互换、长度/自我偏好校正、一致性检验，低一致性时自动增加评委或转人工仲裁。依据：MT-Bench 的"LLM 评委与人类偏好一致率超过 80%、与人类-人类一致率同量级"（arXiv:2306.05685）；G-Eval 的 CoT + 表单式评分（arXiv:2303.16634）；但位置偏差（arXiv:2305.17926）与自我偏好（arXiv:2404.13076）真实存在，必须用换序双评与跨家族评委消除。

创新点 3：成本感知的自适应编排（Token is a Budget）

把 token 当作一等公民：Token Ledger 全链路计量、预算分档、边际收益驱动的候选数量选择、级联评审、早停与剪枝。系统能回答"再迭代一轮值不值"，并在预算耗尽前给出当前最优解。依据：级联可显著降本（FrugalGPT，arXiv:2305.05176）；路由可在质量不降的前提下降低 2 倍以上成本（RouteLLM，arXiv:2406.18665）；缓存与批处理是厂商已提供的确定性折扣（OpenAI Prompt Caching）；而"反复用评委分数选优会导致代理指标先升后降"（arXiv:2309.17453）说明必须早停并把可执行验证混入目标函数。

创新点 4：双形态架构（Orchestrator + Skill）与单一事实源

解决用户原始想法 3 中"头脑与身体脱节"的问题：执行端用 Skill 做单会话自迭代，编排器做跨模型决策，二者通过 SRD/Evidence 共用同一份事实，避免"外部 LLM 瞎指挥、执行模型闷头干"。协议层可直接站在成熟标准上：OpenAI function calling 负责单次工具调用（OpenAI），MCP 负责工具/资源接入（MCP 架构），A2A 负责 Agent 间任务委派（a2a-protocol.org），Agent Skills 负责能力打包分发（Anthropic Skills），而不是自造协议。

创新点 5：机机联动协议（IAOV）与执行体协同

把"工具调用"升级为"可审计的多机闭环"：Intent / Action / Observation / Verdict 四类消息，配合 SRD / Execution Contract / Evidence Packet / ScoreCard 四个状态对象；每次联动都带 contract_hash、权限范围、预算与 trace_id。L0 单机回读已在 demo-004 实测，L1/L2 是设计目标（3.11/7.6），不冒充已实现。与 MAD 的边界是：不做开放式辩论，做有证据、有预算、有权限边界的联动。

6.2 与相关产品/方案的差异

类别	代表产品/方案	它们擅长什么	它们的空位	PromptForge 的差异
提示词优化器	PromptPerfect（Jina AI）等单点工具	把一句话改写成更详细的提示词	不知道执行结果，无证据、无多方案对比；单点工具易被平台能力吞并	用执行证据闭环，多候选假设对比
提示词管理/观测	PromptLayer、LangSmith	版本管理、调用日志、评测数据集	面向"已有提示词"的观测，不解决"从关键词到方案"的决策	从需求编译开始，产出可交付方案与决策报告
模型网关/聚合	OpenRouter、LiteLLM、OneAPI/New API	统一接口、转发、计费、路由	只转发不决策	网关之上有编译、竞演、评审、决策（网关可直接复用成熟方案，不自研）
Agent 应用平台	Coze/扣子、Dify、华为云 AgentArts	低代码搭 Agent/工作流，有编排与基础观测	单一路径执行，缺"多候选同题竞演 + 可审计 MCDM 决策"	多方案并行 + 评审团 + 统计决策 + 交付包
编码 Agent	GitHub Copilot、Cursor、Windsurf、Devin、华为云码道	在 IDE/云端完成编码任务	用户只拿结果，看不到"多模型同题对比 + 评分证据"，返工靠人	可作为它们的"上游方案编译器 + 下游评审台"，也可 Skill 内嵌
模型排名/评测	LMArena、Braintrust	模型排名、应用评测	只给分数与排名，不闭环到"为当前任务交付最佳方案"	从"评测"走到"决策 + 交付 + 复盘"
多智能体辩论框架	各类 MAD 实现	多角色讨论提升部分推理任务	成本约 6~9 倍调用量，等算力下收益缩水；易跑题	不做无限辩论，做有预算、有证据、可复现的决策

差异化一句话：别人优化"提示词"，我们优化"从需求到方案的决策质量"；别人比"模型分数"，我们比"可执行证据 + 成本 + 置信度"。

6.3 关键设计决策与技术依据对照

设计决策	技术依据	对方案的约束
先写验收标准，再生成提示词	APE 在 24 个任务中 19 个达到/超过人工指令（arXiv:2211.01910）、OPRO 比人工 prompt +8%（GSM8K）/+50%（BBH）（arXiv:2309.03409）、DSPy 编译 +25%/+65%（arXiv:2310.03714）——这些方法都以"可自动打分的验证集"为前提	SRD 的验收标准必须可执行；没有验证集时不能宣称"自动优化"

设计决策	技术依据	对方案的约束
多候选并行 + 确定性验证，而非多智能体辩论	辩论有提升但成本约 6~9 倍（arXiv:2305.14325）；等算力预算下未必优于 self-consistency（Should we be going MAD?）	默认并行候选 + 验证器；辩论仅用于高风险子项且受预算约束
多评委 + 换序 + 跨家族 + bootstrap CI	LLM 评委与人类一致率 >80%（arXiv:2306.05685），但存在位置/长度/自我偏好偏差（arXiv:2305.17926、arXiv:2404.13076）	任一评委不得评自己生成的候选；分差落在 CI 内不强行排序
早停 + 验证信号混合	代理指标优化会先升后降（arXiv:2309.17453）	迭代最多 3 轮；测试/编译等硬信号优先于评委分
网关复用而不自研	LiteLLM 支持 100+ 模型统一为 OpenAI 格式 + 路由/预算/成本追踪（GitHub）；RouteLLM 证明路由降本 2×+（arXiv:2406.18665）	自研聚焦上层决策，网关用成熟开源或本地推理
结构化输出 + 工具调用	OpenAI Structured Outputs 保证 schema 合法（OpenAI）；但 schema 不保证语义正确	证据必须来自执行与测试，评委不得只看 JSON 格式
长上下文压缩	LLMLingua 最高 20× 压缩（arXiv:2310.05736）	对超长证据包做摘要/压缩后再给评委，压缩率与保真度需实测

6.4 与"多智能体辩论"的边界

用户原始想法 2 接近"让多个 LLM 来回讨论、互相审查"，这在学术上对应 Multi-Agent Debate。我们的取舍是：**不做开放式辩论，做"并行竞演 + 确定性验证 + 统计评审"**。原因：

1. 辩论的成本随 agent 数 × 轮数线性放大（3 agents × 2~3 轮 ≈ 6~9 倍调用量），而等算力预算下收益可能被 self-consistency 追平（arXiv:2311.17371）；
2. 辩论容易"跑题/趋同"，而代码任务的真相在测试与编译器里，不在讨论里；
3. 比赛与产品都需要可复现、可解释的决策过程，辩论的中间状态难以审计。

因此辩论仅作为**高风险子项的可选环节**（如架构选型争议），且必须受预算与轮数控制。

6.5 与 openJiuwen 生态的关系

openJiuwen 已经提供了完整的 Agent 运行时与工具链，这正是本作品的机会——**不自建重复能力，而是补上"方法层"**：

openJiuwen 能力（官方 FAQ/官网口径）	我们的复用方式	我们不重复做的事
WorkSwarm：任务规划、自演进、上下文瘦身、记忆随行、Channel（小艺/飞书）	作为 Agent 运行与入口层；用其 Skills 机制承载轻量 Skill 形态	不自研 Agent OS、不重写渠道接入
Agent Framework：多智能体协同、自演进、记忆子系统、算力亲和调度	用多 Agent 协同承载候选生成/执行/评审/决策；用记忆子系统沉淀项目知识	不自研调度引擎

openJiuwen 能力（官方 FAQ/官网口径）	我们的复用方式	我们不重复做的事
Agent Studio：低代码工作流、Prompt 调优、在线调试	复用 Prompt 调优作为候选生成策略之一；用工作流编排需求编译与评审流程	不重复做 Prompt 调优
Agent System Service / Agent Runtime：安全隔离沙箱、统一记忆、CLI 工具、标准化文件系统、跨 Agent 通信	执行沙箱与工具调用的底座	不自研沙箱与运行时
DeepAgents / JiuwenSwarm：多实例协同、DeepSearch 等	高风险子项的协同辩论（受预算控制）、资料检索	不做通用的多智能体框架
Swarm Skills Hub	Skill 形态的分发与复用	不自建技能市场

我们的差异化（选题意义上的"补位"）：openJiuwen 生态有"执行与协同"，但"从关键词到可审计决策"的方法层仍缺——**需求编译（SRD）、证据协议（Evidence）、异构评审去偏、TOPSIS/Pareto 决策、Token 预算调度**这五件是我们的自研重点，也是"技术方案"维度最能拿分的地方。这个分工在 §3.9 有组件级说明。

结论【推断】：对评审而言，这既证明"真的用了生态"（拿生态加分），又证明"不是套壳"（有自研方法层）；两者结合比"全部自研"或"全部套壳"都更稳。

本节小结：PromptForge 的新颖性不在"用了 LLM"，而在**把提示词工程升级为可验证、可决策、可计费的工程流程**；技术依据充分、边界清晰（辩论改为竞演、证据优先、网关复用不自研、生态复用不套壳），对比赛而言是一个"技术深度 + 应用价值 + openJiuwen 生态"都能讲通的故事。

7. 实施计划与提交物

> 赛程以官方页面/FAQ 为准，本文按**最保守口径**倒排：10-31 15:59 前完成平台报名+实名+组队，11-10 前提交创意 PPT，12-27 24:00 前提交 DEMO 四件套。

7.1 团队角色与分工（1~5 人，建议 4~5 人）

角色	职责	关键产出	人数
队长/产品	需求编译规则、SRD 模板、创意 PPT、答辩、整体节奏	创意 PPT、演示脚本、答辩 PPT	1
算法/后端	候选生成、评审聚合、TOPSIS/预算调度、成本模型	决策引擎、评分服务、成本账单	1~2
平台/后端	模型网关、沙箱、证据采集、openJiuwen WorkSwarm 集成	Gateway、Sandbox、Evidence API	1~2
前端/可视化	对比看板、会议模式、雷达图/Pareto、成本账单	Web 工作台	1
文档/测试	评测任务集、对照实验、技术报告、视频剪辑、开源仓库	评测报告、技术报告、演示视频	1（可由队长兼）

7.2 倒排里程碑（以 2026-10-20 为起点）

时间窗	里程碑	交付物	验收标准（可判定）
10-20 ~ 10-26	S0 报名与最小闭环	华为云账号实名认证、平台报名、组队；SRD/Evidence/Rubric 三个 Schema；**可运行端到端最小闭环**（1 个模型 + 2 候选 + 规则评审 + 1 页结果）	本机跑通一次"关键词→候选→执行→评分→推荐"并录屏 ≥1 分钟
10-27 ~ 10-31	S1 冻结创意与平台提交	创意 PPT v1（对齐官方模板）、提交材料打包、平台报名/组队完成（硬截止 15:59）	平台显示报名/组队成功；PPT 通过内部评审
11-01 ~ 11-10	S2 创意提交	官方模板 PPT 定稿（含 DEMO 截图/录屏）、zip ≤500MB、命名规范；**11-10 前完成提交（≤5 次上传）**	提交成功回执；PPT 四项齐全（创意名称/描述/主题/场景）
11-11 ~ 11-25	S3 等评审 + 开发 M2	多评委 + 一致性检验 + TOPSIS + 对比看板；技术报告骨架	3 个模型接入；评委 α ≥ 0.6；推荐与人工评审一致率 ≥ 60%
11-25 ~ 12-10	S4 DEMO 功能完整	多候选、多模型、成本账单、会议模式、Skill 形态（至少 1 个）	10 分钟 Demo 一次跑通；成本计量误差 < 5%
12-11 ~ 12-20	S5 材料与开源	3 分钟演示视频、答辩 PPT、技术报告、Apache-2.0 仓库（README/依赖声明/部署指南）	视频 ≤3 分钟；仓库可构建复现；第三方代码占比 ≤30%

时间窗	里程碑	交付物	验收标准（可判定）
12-21 ~ 12-27	S6 回归与提交	全量回归、演示彩排、DEMO 四件套打包（≤5GB，以官方《提交格式说明》为准）	演示环境双备份；**12-27 24:00 前提交成功**
12-28	DEMO 评审	社区专家线上评审	准备 5 分钟讲解 + 10 分钟问答
2027-01 底	决赛路演（30 支）	路演 PPT + 现场 Demo	30/30 秒版本 + 兜底录屏

每周节奏：周一确定本周唯一目标与验收标准；周五演示增量 + 5 分钟复盘。任何功能必须能在演示环境跑通才算完成。

7.3 提交物清单（创意阶段 / DEMO 阶段）

7.3.1 创意阶段（截止 11-10）

材料	要求（事实）	注意
创意 PPT	建议使用**官方模板**；含创意名称、创意描述（<500 字）、所属主题（Coding）、应用场景（<500 字）、方案详情、技术依赖、方案优势、预期 Demo 形式、关键用户路径、示意图	模板附录还有 1–5 分评分量规，见附录 D
打包格式	**zip，≤500MB**	不要把依赖/构建产物打进去；只放 PPT + 必要补充材料
命名	**【openJiuwen 创新大赛创意作品】deforoout 队**	deforoout 为队伍名，提交前核对队名
上传次数	每阶段最多 **5 次**（以最后一次为准）	前 4 次可用于演练，第 5 次前必须定稿
生态说明	如使用 openJiuwen 生态，须说明使用了哪些组件/能力	本文 §3.9 可直接改写为 PPT 的"技术依赖"页

7.3.2 DEMO 阶段（截止 12-27 24:00）

材料	要求（事实）	我们的准备
可运行 DEMO	功能完整的原型	WorkSwarm/本地部署 + 缓存回放兜底
演示视频	**3 分钟以内**	按 4.4 的 10 分钟脚本压缩为 3 分钟：问题→方案→演示→数据→价值
答辩 PPT	决赛/评审用	问题-方案-证据-价值-生态-合规
技术报告/项目文档	设计文档、架构图、测试用例、部署说明	本文档即技术报告底稿
打包	zip（页面 ≤500MB；FAQ/平台配置写≤5GB，以官方《提交格式说明文档》为准）	提前确认，按更严格者准备

7.4 开源与合规计划（获奖强制项，提前做）

事项	要求（事实）	行动
开源协议	**Apache-2.0**，仓库根目录 LICENSE	第一天就建仓、选协议、写 README
公开内容	主体源代码（Agent 编排、工作流、Prompt 工程、工具调用、业务逻辑）、可构建可复现、关键模块注释；代码截图不算开源	从 S0 起保持代码可运行、注释完整
文档	README（名称/介绍/架构/场景/依赖/部署/快速上手）、技术方案（设计文档、架构图、测试用例、演示说明）、第三方依赖声明（名称/版本/来源/许可证/作用/占比）、3 分钟视频、部署运行指南	里程碑 S5 集中补齐，但依赖清单随开发更新
Git 历史	连续、清晰、可追溯	禁止最后一次性导入；每功能一次有意义的提交
第三方代码	**≤ 有效代码量 30%**，不得混入商业闭源代码	依赖全部走包管理器，不 vendor 大段代码；自研核心保持独立
数据合规	来源/许可证/获取/清洗/规模/使用范围声明；禁止未授权爬取、未脱敏个人信息；不得用大赛评测集训练/调参/设计提示词	评测集自建，数据来源写进技术报告
演示安全	不泄露密钥、内部接口地址、真实用户数据；演示数据脱敏模拟	Demo 专用数据 + 环境变量占位
原创性	核心架构/编排/算法/业务逻辑独立完成；可证明独立开发过程	保留设计文档迭代、开发日志、测试演化记录

7.5 风险与缓冲

风险	概率	影响	应对
错过 10-31 报名/组队硬截止	中	**致命**	本周内完成实名+报名+组队；设置提醒
创意阶段只有 PPT、无可运行证据	中	高（创意评审"项目运行"档低分）	10-26 前必须跑通最小闭环并录屏
官方时间口径不一致	高	中	一律按最早截止时间行动（见 1.3.2）
多模型 API 不稳定/涨价	中	中	至少 3 家 + 本地开源模型兜底
评审噪声导致效果不显著	中	高	提高可执行证据权重；准备人工评审对照
团队时间冲突	中	高	每周只承诺一个可演示增量；文档与代码同步
开源合规不达标	低	**致命（取消获奖资格）**	提前建仓、协议/依赖/历史三件套随开发维护

砍需求顺序（时间不够时从后往前砍）：会议模式批注 → 融合与回归 → 多模型（保 2 家）→ 可视化（保雷达图+排名）→ **最小闭环不可砍**。

本节小结：比赛真正的胜负手是"**11-10 前拿得出可运行证据、12-27 前拿得出稳定可复现的 DEMO + 合规开源仓库**"；因此计划以"报名/创意/DEMO 三个硬截止"为轴，所有技术增强项都为这两个交付节点服务。

7.6 机机联动的演进路线（设计目标，不是当前承诺）

> 本节只放在设计文档，不放进创意 PPT：创意阶段先讲清"是什么、为什么值得做、第一段已经跑通"，避免用未来计划撑门面。

层级	目标	关键能力	验收标准（目标值，需实测校准）	依赖
L0 单机回读	执行 → 证据 → 决策	pytest/evidence/ decision	已由 demo-004 验证（A 3/6、B 6/6，可复现）	本机 Python + DeepSeek
L1 跨进程联动	规划机/执行机/评审机分工	IAOV、MCP/A2A、预算/权限、trace	一次任务 ≥2 个执行端点；证据完整率 ≥90%；可回放	WorkSwarm / Agent Framework / Agent System Service
L2 跨环境联动	CI、浏览器、IDE 端点	Capability Manifest、幂等、环境指纹、回放	3 类端点可替换；回放一致率 ≥80%	Agent System Service + CI/浏览器适配器
L3 设备联动	设备/机器人安全闭环	安全包络、人工确认、急停、受限动作	受控场景完成 1 个端到端任务；零越权	硬件/仿真环境

推进原则：

1. **先证据、后设备**：L1 的验收标准不是"接入了几个模型"，而是"证据完整率、回放一致率、单位采纳方案成本"；
2. **单层可停止**：每层都必须有预算上限、轮数上限、权限撤销与人工确认，不能因为"联动"绕过安全边界；
3. **不承诺时间表**：L2/L3 的排期取决于 L1 的实测结果与可用硬件，不在创意材料中承诺日期；
4. **与主线解耦**：机机联动是 3.11 的扩展层，不影响 v0.1 已跑通的"编译 → 竞演 → 证据 → 决策"主线；即使 L2/L3 延期，作品仍可交付。

8. 答辩预演：评委可能问的问题

8.1 高频问题与回答要点

Q1：这和"让大模型自己优化提示词"有什么区别？

A：单模型的 self-refine 是"同一条思路上的自我修改"，容易陷入局部最优且没有外部证据；PromptForge 是"多个不同假设的方案并行执行 + 用测试/指标取证 + 异构评委盲评 + 统计决策"，本质是把"提示词优化"变成"方案决策"。我们不做无限自我反思，做有预算、有证据的择优。

Q2：多个 LLM 评审可靠吗？会不会互相吹捧或随便打分？

A：不可靠的单评委确实有偏差，所以我们做了四件事：评委异构（不同厂商/规模）、匿名盲评 + 位置互换、长度/自我偏好校正、一致性检验（Krippendorff's α，低于阈值自动加评委或转人工）。同时把"可执行证据"作为强信号，LLM 评分只占部分权重。最终输出带置信区间，而不是一个绝对分数。

Q3：多方案多评委，成本会不会翻好几倍？

A：在 3.7 的算例参数下，朴素实现约为单轮的 6 倍以上；但更公平的对照是"单模型 self-refine 长跑"，算例中我们约为它的 2.16 倍；再叠加候选剪枝、弱评委、缓存、早停，经济档在算例中可压到单轮的 3~5 倍（约长跑的 1.2~1.7 倍）。上述均为【算例】或【推断】，实际倍数需用真实报价与实测 token 校准（见附录 G）。系统会算"再迭代一轮值不值"，在预算耗尽前输出当前最优；对低价值任务默认走 Skill 单机模式，成本约为单次调用的 1.2~2 倍（估算）。

Q4：为什么用 openJiuwen 生态？是不是套壳？

A：我们复用的是它已经成熟的"运行时/沙箱/协同/渠道/技能分发"（WorkSwarm、Agent Framework、Agent Studio、Agent System Service），自研的是它当前不覆盖的"决策层"：需求编译（SRD）、证据协议、评审去偏、TOPSIS/Pareto 决策、Token 预算调度。生态是地基，方法是壁垒——这既符合官方"用生态加分"的规则，也不是套壳。组件级说明见 3.9。

Q5：你们怎么保证评审数据与效果数据不注水、不造假？

A：三条机制：① 所有指标在实验前登记（指标定义、任务集版本、模型版本、参数），不做"看到结果再改指标"；② 每次评分保留原始日志（模型、token、价格、耗时、评委原始分、位置顺序），交付包可回放；③ 对外只引用台账中已实测的数据，标【算例】【推断】【设计目标】的绝不写成"实测结果"；DEMO 阶段提供可复现实验脚本与原始记录，欢迎评委抽查。数据台账见附录 G。

Q6：图像/视频这类多模态任务怎么评审？

A：不把原始媒体直接送入模型，而是走"多模态证据通道"：关键帧/截图交多模态模型理解；OCR/ASR/ffprobe/UI 自动化等本地工具提取结构化字段（字幕、元数据、界面差异）；结构化结果写入

Evidence Packet，作为"中证据"参与评审。代码类任务仍以测试、编译、静态分析等强证据为主。该能力为可选模块，默认关闭，仅在任务确实涉及 UI/图表/演示内容时启用（详见 3.3.4）。

Q7：如果所有候选都很差怎么办？

A：先做"候选预评审门禁"，避免用贵流程筛烂提示词；若全部不合格，回到需求编译，检查 SRD 是否不可执行或目标过高；必要时降级为 MVP 拆解，而不是继续堆提示词。系统会把失败原因写入交付包，方便复盘。

Q8：融合多个方案的优点，会不会变成"缝合怪"？

A：会，所以我们只融合低耦合优点（测试用例、边界处理、文档、提示词片段），架构级融合放二期；融合后用 McNemar 检验做显著性判断，不显著就不宣称融合更好，并强制全量回归测试，不通过就回退到最优单方案。融合不是必选项。

Q9：和 Cursor/Copilot/Devin 是什么关系？

A：它们是把任务做出来的"执行器"，我们更像执行器上层的"方案编译器 + 评审台"。我们可以把优化后的提示词与验收标准交给它们执行，也可以以 Skill 形式内嵌；不替代 IDE，而是让 IDE 里的 AI 一开始就方向正确。

Q10：商业价值在哪里？

A：三层：个人开发者按次/订阅（低成本档）；企业团队私有化部署 + 内部规范知识库（高客单）；API/平台化（把"方案决策"能力输出给其他 Agent 平台）。核心指标是"减少的返工时间与 token 成本"，可直接折算 ROI。

Q11：你们怎么证明效果？

A：30 个任务的 A/B 对照：传统方式（直接问一个大模型 + 人工来回改）vs PromptForge 一次交付，记录返工轮数、总 token、耗时、人工评分；同时报告评委一致性与推荐采纳率。所有数字来自实际记录，答辩时展示原始日志。

Q12：获奖后要开源，会不会被人抄？

A：比赛规则要求获奖作品 Apache-2.0 开源，我们接受这个规则并把壁垒放在"数据闭环 + 团队知识库 + 方法迭代"上，而不是代码保密；开源反而有利于生态集成与获客。

Q13：最大的技术风险是什么？

A：评审噪声与融合回归。应对：提高可执行证据权重、异构评委 + 一致性门槛、融合强制回归 + 回退策略。我们宁愿"保守地推荐一个稳的方案"，也不"激进地推一个没验证过的融合体"。

8.2 三类叙事版本

- **30 秒版**：输入一句话，PromptForge 生成多个不同假设的方案，真正跑起来、拿证据、由多个不同厂商的模型盲评，最后用数学模型选出最优并给出成本账单——让 AI 开发从"反复试错"变成"一次选对"。
- **3 分钟版**：在 30 秒版基础上加"为什么难"（提示词质量方差大、单模型自评有偏差、多方案成本高）与"我们怎么解"（SRD 编译、证据回读、异构评审、预算调度、双形态、openJiuwen 生态）。
- **电梯版（一句话）：PromptForge 是 AI 开发者的"方案编译器 + 评审会"，让每个开发者都请得起一个由多个模型组成的专家评审团。**

本节小结：答辩的核心不是证明"我们技术最先进"，而是证明"我们想清楚了边界"——知道什么可行、什么不可行、成本多少、风险在哪、生态用什么补什么，并且有可演示的闭环与真实数据。

附录 A：数学模型与算例摘要

> 本附录所有数值均为【**算例**】（人为构造的参数与结果），只用于演示公式与计算步骤，**不代表任何实测结果或报价**；落地参数须替换为实际值并登记在附录 G 数据台账。完整推导与 22 条自洽性校验见调研笔记 `research/eval-cost.md`。

A.1 TOPSIS 算例（4 候选 × 5 指标，权重取均衡预设，数值人为构造）

原始矩阵与计算步骤见 3.5.2。关键中间结果：

候选	向量归一化后正确性	加权后成本	D^+	D^-	贴近度 C^*	排名
A	0.4927	0.0918	0.042011	0.046055	0.522960	3
B	0.5408	0.1257	0.069249	0.046328	0.400841	4
C	0.4447	0.0628	0.053533	0.069303	**0.564189**	**1**
D	0.5168	0.1088	0.049675	0.054979	0.525343	2

结论：C > D > A > B；但 A 与 D 差 0.0024（0.46%），低于噪声，须 bootstrap 秩稳定性或并列进下一轮；C 若正确性硬门槛 ≥80 则先被过滤。

A.2 Pareto / HV 与 SAW 对比

- **HV**：非支配集 {E(30,0.80), G(40,0.84), F(45,0.86), H(55,0.88)}，参考点 (60,0.75)，HV=2.70；新增 K(42,0.87) 支配 F 后 HV'=2.89，**ΔHV=+7.04%**。
- **SAW vs TOPSIS 反例**：X=(10,10,1)、Y=(6,6,6)，等权 3 指标；SAW 归一化 X=0.667 > Y=0.333，TOPSIS C_X=0.371 < C_Y=0.629——两法可给出相反排序，方法选择必须写明价值取向。

A.3 评审聚合算例

项目	数据	结论
去极值	4 评委对 A/B/C 打分，J3 给 B 离群 5.0	原始均值 A 8.275 > B 8.000；去极值后 B 8.900 > A 8.300，**排序反转**
Bradley-Terry	4 候选 × 每对 8 场胜场	θ=(A 0.4952, C 0.2125, B 0.1858, D 0.1064)；P(A>B)=0.7272；P(C>B)=0.534 → B/C 属弱排序
Elo	初始 1500，K=32；4 评委 3:1	A +8 → 1508；B −8 → 1492；再胜一场 A +15.3 → 1523.3
Krippendorff α	5 单元 2 评委名义值	α=0.25 < 0.667，一致性不可用
Cohen κ	20 项通过/不通过	κ=0.375，仅 fair，不足以支撑高 stakes 决策

A.4 成本算例（假设参数，仅演示计算过程）

组成	金额	占比
轮 1（N=3）：生成 0.0336 + 执行 0.5100 + 评审 0.0672	¥0.6108	53.8%
轮 2（N=2）：生成 0.0224 + 执行 0.3400 + 评审 0.0448	¥0.4072	35.8%
融合	¥0.0220	1.9%

组成	金额	占比
回归验证	¥0.0962	8.5%
合计	**¥1.1362**	100%
单模型单轮基线	¥0.1756	6.47×
单模型 self-refine 3 轮	¥0.5268	**2.16×**（更公平的对照）

优化变体：剪枝到 N_2=1 → ¥0.9326（−17.9%）；前缀缓存 → ¥0.9602（−15.5%【推断】）；弱评委 → ¥1.0507（−7.5%）；弱评委+剪枝 → ¥0.8642（−23.9%）。

A.5 停止准则算例

方法	关键数字	结论
Beta-Bernoulli	P(A best)：R1 0.787 → R3 0.926 → R4 0.980	R4 达 0.95 阈值，停止
SPRT	边界 ln16=2.7726；n=16, k=13 时 Λ=3.19	停止接受"挑战者更优"；期望样本量约 16.4
边际收益	第 3 轮 V·ΔQ=0.30 vs 成本 0.28；第 4 轮 0.16 vs 0.25	跑完第 3 轮即停

A.6 自洽性抽查（可直接验算）

#	校验	结果
1	TOPSIS C：0.046055/(0.042011+0.046055)	0.522960 ✓
2	HV：10×0.05 + 5×0.09 + 10×0.11 + 5×0.13	2.70 ✓
3	BT 归一化：0.4952+0.1858+0.2125+0.1064	1.0000 ✓
4	κ：(0.75−0.60)/(1−0.60)	0.375 ✓
5	α：1 − 0.4/0.5333	0.25 ✓
6	成本轮 1：0.0336+0.5100+0.0672	0.6108 ✓
7	成本合计：0.6108+0.4072+0.0220+0.0962	1.1362 ✓
8	倍数：1.1362/0.1756；1.1362/0.5268	6.47×；2.16× ✓
9	SPRT：ln(16)=2.7726；n=16,k=13 → 3.1916	≥ 边界，停 ✓
10	剪枝+弱评委：R1(0.0336+0.51+0.0159)+R2'(0.0112+0.17+0.0053)+0.022+0.0962	0.8642 ✓

附录 B：技术依据与参考来源

> 抓取/核对日期：2026-10-20。论文给 arXiv 摘要页；产品页面会变化，引用以官网为准。标注【未核对】的仅给量级，正式引用前复核。

B.1 竞赛与生态

- 官方赛事页：https://developer.huaweicloud.com/competition/information/1300000328/introduction
- 提交页：https://developer.huaweicloud.com/competition/information/1300000328/submission ；组队页：`.../myteam`
- 官方参赛 FAQ（2026-10 版，25 页）与官方创意 PPT 模板（中/英）：官方赛事页内下载（本机副本 `_build/api/FAQ.pdf`、`template_CN.pptx`）
- openJiuwen 官网 / WorkSwarm：https://openjiuwen.com/ ；https://openjiuwen.com/workswarm
- 华为云 AI 开发者空间：https://devstation.developer.huaweicloud.com/aidevspace?port=9000
- 赛事交流区：https://gitcode.com/org/openJiuwen/discussions/830

B.2 提示词自动优化

- APE：Zhou et al., 2022，arXiv:2211.01910
- OPRO：Yang et al., ICLR 2024，arXiv:2309.03409
- DSPy：Khattab et al., 2023，arXiv:2310.03714
- TextGrad：Yuksekgonul et al., 2024，arXiv:2406.07496
- PromptBreeder：arXiv:2309.16797
- EvoPrompt：arXiv:2309.08532
- Self-Refine：arXiv:2303.17651
- Reflexion：arXiv:2303.11366
- LLMLingua：arXiv:2310.05736 / arXiv:2403.12968

B.3 LLM 评审与验证

- G-Eval：arXiv:2303.16634
- MT-Bench / Judging LLM-as-a-Judge：arXiv:2306.05685
- 位置偏差：arXiv:2305.17926
- 自我偏好：arXiv:2404.13076
- Chatbot Arena / Bradley-Terry：arXiv:2403.04132
- Multi-Agent Debate：arXiv:2305.14325
- 辩论公平比较（Should we be going MAD?）：arXiv:2311.17371
- 代理指标过度优化：arXiv:2309.17453

- 采样与验证器（Large Language Monkeys）：arXiv:2407.21787
- SWE-bench：arXiv:2307.03172
- CodeT：arXiv:2207.10397

B.4 协议、网关与成本

- OpenAI function calling：https://openai.com/index/function-calling-and-other-api-updates/
- OpenAI Structured Outputs：https://openai.com/index/introduction-to-structured-outputs/
- MCP 架构：https://modelcontextprotocol.io/specification/2026-07-28/architecture/index.md
- A2A：https://a2a-protocol.org/latest/
- Anthropic Agent Skills：https://claude.com/blog/building-agents-with-skills-equipping-agents-for-specialized-work
- LiteLLM：https://github.com/BerriAI/litellm ； One API：https://github.com/songquanpeng/one-api ； New API：https://github.com/QuantumNous/new-api ； OpenRouter：https://openrouter.ai/
- RouteLLM：arXiv:2406.18665
- FrugalGPT（级联）：arXiv:2305.05176
- vLLM：arXiv:2309.06180
- OpenAI Prompt Caching：https://platform.openai.com/docs/guides/prompt-caching
- Claude Prompt Caching：https://platform.claude.com/docs/en/build-with-claude/prompt-caching
- DeepSeek 模型与价格（2026-10-20 抓取）：https://api-docs.deepseek.com/zh-cn/quick_start/pricing/
- OpenAI Vision：https://platform.openai.com/docs/guides/images-vision ； Claude Vision：https://platform.claude.com/docs/en/build-with-claude/vision

B.5 多准则决策与统计方法

- TOPSIS：Hwang & Yoon (1981), *Multiple Attribute Decision Making*
- Bradley-Terry：Bradley & Terry (1952)；MM 算法：Hunter (2004)
- Elo：Elo (1978), *The Rating of Chessplayers*
- Krippendorff's α：Krippendorff (2004), *Content Analysis*
- Cohen's κ：Cohen (1960)；ICC：Shrout & Fleiss (1979)
- SPRT：Wald (1945)
- UCB1：Auer, Cesa-Bianchi & Fischer (2002)；Thompson Sampling：Thompson (1933)
- McNemar (1947)

B.6 价格与事实标注

- DeepSeek 官方价（2026-10-20 抓取，可能随时间变化）：deepseek-flash 缓存命中输入 0.02–0.04 元/M、未命中 1–2 元/M、输出 4–8 元/M（空闲半价）；deepseek-v4-pro 0.15–0.30 / 4.5–9.0 / 13.5–27 元/M；上下文 1M。来源：https://api-docs.deepseek.com/zh-cn/quick_start/pricing/
- 其他厂商（OpenAI/Anthropic/阿里/火山）价格请以官网实时报价为准；本文不引用未经核对的具体数值。
- 华为云产品与 openJiuwen 组件信息以 2026-10 官网口径为准，产品迭代快，引用前复核。

附录 C：官方创意 PPT 逐页大纲（创意提交版）

> 官方模板共 7 页（P1–P7）；本作品提交版保留 **P1–P4 + 2 页附页 = 6 页**，不保留官方 P5/P6 参考页，不设"未来计划"页。未来规划只放在设计文档 §7.6，不进入创意 PPT。

P1 封面

- 创意名称：**PromptForge（提示词锻造台）**
- 一句话：**让 AI 开发从"反复试错"变成"一次选对"**
- 副标签：提示词编译器 · 多智能体评审会 · 机机联动闭环
- 团队 / 赛道：deforoout 队 | Coding 赛道 | 首届 openJiuwen 智能体创新大赛

P2 创意名称 / 创意描述 / 主题 / 场景

- **定位**：PromptForge 是 AI 开发者的"方案编译器 + 多智能体评审会 + 机机联动执行网络"。
- **创意描述（<500 字）**：输入一句话 → 编译成含 6 条可执行验收标准的 SRD → 生成 3~5 个"假设不同"的候选提示词 → 多模型真实执行并回读证据 → 异构评委盲评 → TOPSIS/Pareto 选出最优 → 输出证据包、成本账单与可回放交付包。解决"需求说不清、方案选不对、结果没验证"。
- **所属主题**：Coding 赛道。
- **应用场景（<500 字）**：独立开发者、高校学生、2~20 人研发团队；从一句话想法做原型、给现有项目加功能/修 Bug、技术方案选型与团队评审。
- **真实证据**：同一 deepseek-flash，A raw 3/6 vs B compiled 6/6；费用 ¥0.0096 vs ¥0.0055（runs/demo-004，2026-10-27）。
- **和现有方案的区别**：提示词工具只改文本；单模型 self-refine 没有外部证据；多智能体辩论成本不可控。PromptForge 用执行证据做决策。

P3 创意方案构想

- **七阶段流水线**：需求编译 → 候选组合 → 多模型执行 → 证据回读 → 异构评审 → 决策融合 → 交付回放。
- **三类贯通状态**：SRD（要什么） | Evidence Packet（机器做成了什么） | Prompt DAG（怎么演化来的）。
- **核心机制**：证据驱动、异构评委去偏、Token 预算、Orchestrator + Skill 双形态。
- **技术依赖**：openJiuwen WorkSwarm/AgentOS、Agent Framework、Agent System Service；DeepSeek/Qwen/Pangu；Python + pytest + 沙箱。
- **方案优势**：从"文本优化"升级为"方案决策"；推荐能指到具体测试输出与费用；多候选是假设不同；成本可算、可停止；交付可复现、可审计。
- **机机联动**：在流水线中标注"执行结果回读 → 评审结论再驱动下一轮"，不展开路线图。

P4 DEMO 或示意图 + 关键用户路径

- **左侧**：A/B 候选证据卡（6 条测试逐条红绿、失败用例名、费用、耗时）+ 决策卡。
- **右侧**：关键用户路径：① 一句话输入 → ② SRD 回显 → ③ 生成 2~4 个候选 → ④ 沙箱执行 + 证据回读 → ⑤ 异构评审 + 对比看板 → ⑥ 推荐 + 成本账单 + 交付包。
- **底部**：真实运行信息（runs/demo-004）、交付物、代码仓库、数据规则（不伪造、不手填）。

附页 A：机制展开

- 四象限：需求编译 SRD、候选组合 Portfolio、证据包 Evidence Packet、评审与决策。
- 每个象限给一个真实/具体例子（6 条验收、A/B 候选差异、A 3/6 失败用例、硬约束过滤）。
- 底部：双形态与单一事实源、Token Ledger 预算与早停。

附页 B：机机联动

- 三层机器：规划机 Planner、执行机 Executor、评审机 Judge；中间是 IAOV 联动总线（Intent / Action / Observation / Verdict）。
- 难点：异构接口、权限与安全、状态与成本；对策：Capability Manifest、沙箱 + 最小权限、Token Ledger + 回放。
- 已跑通第一段：生成 → 沙箱 → pytest → Evidence Packet → 决策卡（demo-004 真实数据）。
- 边界：不做无协议、无预算、无权限的开放式多机聊天；高风险动作必须人工确认。
- **不在 PPT 写 L1/L2/L3 时间表或"下一步"**；演进路线见 §7.6。

附录 D：官方评审量规自评表（1–5 分）

D.1 官方模板附录量规（事实：来自官方创意 PPT 模板第 6 页）

维度	1 分	3 分	5 分	我们的目标	支撑证据	补强动作
项目运行	重大问题或未运行	可运行	卓越且安全的架构	**4→5**	可运行原型；沙箱隔离/权限最小化/审计日志	11-10 前录屏；DEMO 现场演示安全架构
创新与创造力	非原创	有些新意	突破性想法 + 出色实施	**4**	证据驱动决策、异构评委去偏、成本感知编排、双形态	用对比表突出机制创新而非模型能力
对现实世界的影响	用途有限	当前有一定影响	改变生活的潜力	**4**	返工减少、门槛降低、团队评审可量化	补 3 个真实用户故事与前后数据
主题一致性	几乎不相关	主题一致	完美体现并提升主题	**4→5**	多智能体协同决策（"智体共生"）；现实场景创新应用	封面第一页点题；演示多 Agent 协作

D.2 创意 / DEMO 阶段维度自检

阶段	维度	我们的回答	目标
创意	创新性	不是提示词改写器，而是可审计的方案决策系统；有论文依据与竞品空位分析	4/5
创意	实用性	减少返工轮数与 token；团队评审从"拍脑袋"到"看数据"	4/5
创意	技术可行性	成熟模块 + openJiuwen 底座 + 8~12 人周计划 + 最小闭环已在路线图中	4/5
创意	场景价值	Coding 最痛、可扩展到办公/科研；个人与团队都受益	4/5
DEMO	完成度	端到端闭环 + 双备份演示环境	4/5
DEMO	用户体验	对比看板/会议模式/决策卡/成本账单；新手 3 问内可用	4/5
DEMO	技术难度	多模型网关、证据协议、BT/TOPSIS、预算调度、Skill/Orchestrator	5/5
DEMO	商业价值	个人订阅/企业私有化/API 平台三层；ROI 可折算	3~4/5

附录 E：提交检查清单

E.1 报名与组队（硬截止 2026-10-31 15:59）

- ☐ [ ] 华为云账号已完成实名认证
- ☐ [ ] 已通过平台报名（.../submission 或官方入口）并组建/加入团队
- ☐ [ ] 队伍人数 1~5，成员未重复组队，全队只提交 1 个作品
- ☐ [ ] 已确认赛道：Coding（可多选方向，但作品说明统一以 Coding 为主）
- ☐ [ ] 已加入赛事交流区（GitCode discussions）并关注官方通知

E.2 创意提交（截止 2026-11-10）

- ☐ [ ] 使用官方模板 PPT（或包含全部必填字段）
- ☐ [ ] 创意名称、创意描述 <500 字、所属主题、应用场景 <500 字、方案详情、技术依赖、方案优势、预期 Demo、关键用户路径、示意图齐全
- ☐ [ ] 已写明使用的 openJiuwen 组件（技术依赖页）
- ☐ [ ] zip ≤500MB；文件命名【openJiuwen 创新大赛创意作品】deforoout 队
- ☐ [ ] 上传次数 ≤5（以最后一次为准），提交后保留回执
- ☐ [ ] 内部交叉检查：无错别字、无夸大未实现功能、无真实密钥/内部地址

E.3 DEMO 提交（截止 2026-12-27 24:00）

- ☐ [ ] 可运行原型（演示环境 + 本地兜底双备份）
- ☐ [ ] 演示视频 ≤3 分钟，数据脱敏
- ☐ [ ] 答辩 PPT
- ☐ [ ] 技术报告/项目文档（本文档扩展版）
- ☐ [ ] zip 大小符合官方《提交格式说明》（页面 500MB / FAQ、平台 5GB，按更严格者准备）
- ☐ [ ] 提交前完整彩排 ≥2 次，录屏兜底

E.4 开源与合规（获奖强制项）

- ☐ [ ] 仓库根目录 LICENSE（Apache-2.0）
- ☐ [ ] README：名称/介绍/架构/场景/依赖/部署/快速上手
- ☐ [ ] 技术方案：设计文档、架构图、测试用例、演示说明
- ☐ [ ] 第三方依赖声明：名称/版本/来源/许可证/作用与占比；第三方代码 ≤30%
- ☐ [ ] 部署指南与环境版本；.env.example 占位，无真实密钥
- ☐ [ ] Git 提交历史连续、清晰、可追溯
- ☐ [ ] 数据合规声明：来源/许可证/获取/清洗/规模/使用范围；无未授权爬取、无未脱敏个人信息
- ☐ [ ] 未使用大赛评测集/测试集训练、调参、设计提示词

☐ [ ] 演示材料与仓库一致，无伪造数据/演示效果

附录 F：术语缩写与变更记录

F.1 术语缩写

缩写	全称	含义
SRD	Structured Requirement Doc	结构化需求规格（目标/约束/验收/非目标）
PIR	Prompt Intermediate Representation	提示词中间表示（8 段式结构）
Evidence	Evidence Packet	证据包（测试/静态分析/性能/截图/成本/可复现信息）
BT	Bradley-Terry	成对比较的强度模型
MCDM	Multi-Criteria Decision Making	多准则决策
TOPSIS	Technique for Order Preference by Similarity to Ideal Solution	逼近理想解排序
HV	Hypervolume	超体积（Pareto 前沿评价）
MAD	Multi-Agent Debate	多智能体辩论
MCP	Model Context Protocol	模型上下文协议（工具/资源接入）
A2A	Agent2Agent	Agent 间协作协议
SPRT	Sequential Probability Ratio Test	序贯概率比检验
UCB	Upper Confidence Bound	多臂老虎机上置信界
CI	Confidence Interval	置信区间
TRL	Technology Readiness Level	技术成熟度等级（1–9）

F.2 变更记录

版本	日期	变更
v1.0	2026-10-20	初稿：完成系统架构、核心机制（需求编译/多候选/证据/评审/决策/融合/成本/双形态/openJiuwen 落地/安全）、可行性评估、创新点、实施计划、答辩预演与附录
v1.1	2026-10-23	删除原稿中不适用内容（媒体输入误区、非官方渠道调用）；新增数据与信息规范、附录 G 数据与假设台账；多模态改为可选证据通道；排版与阅读路径重构，作为开发基线

附录 G：数据与假设台账

> 本附录是"不造假"的执行工具：所有在设计阶段出现的数字必须在此登记性质、来源、替换方式与责任人；没有登记的数字不得写进对外材料。

G.1 数据分级与引用规则

级别	标记	允许的表述	禁止的表述
官方事实	【事实】	"据官方页面/FAQ/接口（日期）……"	脱离来源的绝对化表述
文献结论	引用链接	"论文报告/官方文档口径为……"	把论文数字说成我们的实测
数学算例	【算例】	"算例演示：在假设参数下……"	"测试表明/实测得到"
设计目标	【设计目标】	"目标是……（待 A/B 校准）"	"已经达到……"
推断	【推断】	"基于经验推断……"	当作事实陈述

G.2 数据与假设台账（v1.1 初始版）

ID	参数/数字	当前值/范围	性质	来源或依据	替换/验证方式	责任角色	目标时间
D-01	模型单价（生成/执行/评委）	假设 ¥2/¥8、¥0.5/¥1.5 每百万 token	假设	参照 2026-10-20 DeepSeek 官方价量级	以实际供应商报价替换；价目变更时更新	平台/后端	S0
D-02	单任务 token 画像	生成 800/1200、执行 2500/1500、评分 2200/150	假设	经验估算	用真实调用日志统计 p50/p95	算法/后端	S1
D-03	评测任务集规模	30 条	设计参数	实验设计	按任务类别分层抽样并固定版本	文档/测试	S3
D-04	候选数/评委数/轮数	N=3~5、K=4、T≤3	设计参数	方法论	用成本-质量曲线标定	算法	S4
D-05	成本倍数	6.47×/2.16×/3~5×	算例	3.7 公式推导（假设参数）	用真实价格与 token 重算	算法	S4
D-06	TOPSIS/BT/HV/SPRT 示例数值	4×5 矩阵等	算例	人为构造	仅用于文档演示，不对外称实测	文档	已完成
D-07	目标指标（一次通过率、返工轮数、采纳率等）	见 4.5 表	设计目标	产品设计	A/B 实验实测并记录原始日志	文档/测试	S5
D-08	"辩论成本 6~9 倍""优化节省比例"	区间	推断	文献/经验	实验验证后更新	算法	S5
D-09	竞赛事实（名称/时间/奖金/评审维度/开源要求）	见 1.3	官方事实	官方页面、FAQ、平台接口（2026-10-20，2026-10-23 复核）	提交前每日复核官方通知	队长	持续

ID	参数/数字	当前值/范围	性质	来源或依据	替换/验证方式	责任角色	目标时间
D-10	DeepSeek 官方价格	见附录 B.6	官方事实	DeepSeek 官方定价页（2026-10-20）	使用前重新抓取，注明日期	平台	持续
D-11	文献数字（APE/OPRO/DSPy/MT-Bench 等）	见正文引用	文献结论	论文摘要/官方文档	引用保留原文链接，不二次加工	文档	已完成
D-12	多模态 token 参考量级	1024^2 ≈765/1.4k token	官方文档口径	OpenAI/Claude 视觉文档	以供应商当前文档为准，实测校验	算法	S4
D-13	团队规模与工作量	4~5 人、8~12 人·周	估算	团队计划	每周复盘校正	队长	持续

G.3 实验记录规范

每次评测/对照实验必须留下以下字段（写入 `experiments/` 目录，随仓库提交）：

```
{
  "experiment_id": "exp-YYYYMMDD-序号",
  "date": "YYYY-MM-DD",
  "git_commit": "...",
  "task_set_version": "v0.1",
  "models": [{"provider": "...", "model": "...", "version": "...", "price_snapshot": "..."}],
  "params": {"N": 3, "K": 4, "T": 2, "budget_cny": 0},
  "metrics": {"pass_rate": null, "rework_rounds": null, "cost_cny": null},
  "raw_logs": ["logs/..."],
  "notes": "指标为实测；无数据填 null，不允许填猜测值"
}
```

规则：① 指标定义与阈值在实验前登记（预注册），不得事后更改；② 原始日志只追加、不覆盖；③ 报告中的数据必须能从日志复算；④ 任何"演示专用"数据必须标注且与实验数据分开存放。

G.4 对外表述与变更规则

- 对外材料（PPT/视频/答辩/README）引用数字时，必须同时能指出其台账 ID 与级别（事实/实测/算例/目标/推断）。
- 实测数据只有写入台账并附原始日志后，才允许对外表述为"实测/验证"。
- 文档变更：任何影响范围、成本、指标的修改，需更新台账与修订记录；重大架构决策另记 ADR（见《实施工作台》）。
- 若发现既往表述与数据不符，立即更正并在修订记录中说明，不静默修改。